Universidad Mayor de San Simón

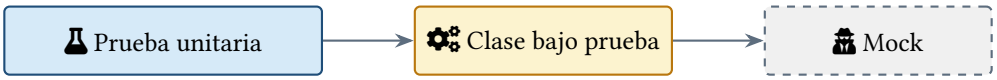
Facultad de Ciencias y Tecnología

Carrera de Ingeniería de Sistemas

Materia: Auditoría de Sistemas

Mockito

Framework de dobles de prueba para Java, especificación de requisitos (SRS), diseño de software (SDD) y diseño paso a paso de pruebas de caja blanca y caja negra



Grupo 10

Main Rocabado Jairo Gael
Banda Tellez Fernando
Velasquez Ricaldez Rodrigo

Exposición: 29 de septiembre de 2026 08:15 – 08:44

Cochabamba – Bolivia 2026

Resumen

En las pruebas unitarias hay que aislar la unidad que se prueba de sus dependencias: bases de datos, servicios web, pasarelas de pago o servidores de correo. **Mockito** es el framework de código abierto más usado en el ecosistema Java para crear *dobles de prueba* (mocks, stubs y spies). Con él, el tester programa el comportamiento de esas dependencias y verifica cómo interactúa con ellas la clase bajo prueba.

Este informe presenta primero el marco teórico: pruebas de software, dobles de prueba y técnicas de caja negra y caja blanca. Luego explica qué es Mockito, cómo se usa su API y, en detalle, su **arquitectura y componentes internos**. Para aplicar la teoría se diseñó un sistema de ejemplo, un *Servicio de Pedidos*, especificado con una **SRS** (*Software Requirements Specification*, IEEE 830 / ISO/IEC/IEEE 29148) y documentado con un **SDD** (*Software Design Description*, IEEE 1016-2009). Se explica paso a paso cómo se diseña cada caso de prueba, desde el requisito hasta el código, y se desarrollaron 65 pruebas automatizadas con JUnit 6 y Mockito 5.24:

- diez ejemplos didácticos que recorren la API de Mockito y hacen visibles sus componentes internos;
- pruebas de **caja negra**: partición en clases de equivalencia, análisis de valores límite y tabla de decisión;
- pruebas de **caja blanca**: camino básico de McCabe con $V(G) = 7$ y cobertura de decisiones.

Las 65 pruebas pasan y alcanzan el 100 % de cobertura de ramas en la lógica de negocio, medida con JaCoCo. Además, la comparación muestra que la caja negra por sí sola cubre 11 de 12 ramas; la rama que falta solo aparece al analizar el código. Así se confirma que las dos técnicas se complementan.

Desde el punto de vista de la **auditoría de sistemas**, las pruebas automatizadas con dobles de prueba producen evidencia repetible, trazable a los requisitos y medible mediante cobertura. Esa evidencia sirve para evaluar la calidad de los controles de desarrollo y cambio del software.

Palabras clave: Mockito, pruebas unitarias, dobles de prueba, caja negra, caja blanca, complejidad ciclomática, diseño de casos de prueba, SRS, SDD, IEEE 1016, JUnit, auditoría de sistemas.

Índice general

Resumen I

1. Introducción 1

1.1. Antecedentes . . . 1
1.2. Objetivos . . . 1
1.2.1. Objetivo general . . . 1
1.2.2. Objetivos específicos . . . 1
1.3. Justificación . . . 2
1.4. Alcance . . . 2

2. Marco teórico 3

2.1. Pruebas de software . . . 3
2.1.1. Características de una buena prueba unitaria (FIRST) . . . 3
2.2. Dobles de prueba . . . 4
2.2.1. Verificación de estado frente a verificación de comportamiento . . . 4
2.3. Pruebas de caja negra . . . 4
2.4. Pruebas de caja blanca . . . 5
2.4.1. Complejidad ciclomática de McCabe . . . 5
2.4.2. ¿Qué aporta Mockito a la caja blanca? . . . 5
2.5. Las pruebas en la auditoría de sistemas . . . 5

3. Mockito 6

3.1. ¿Qué es Mockito? . . . 6
3.2. ¿Cómo funciona? Visión rápida . . . 6
3.3. Instalación . . . 6
3.4. API principal . . . 7
3.5. Estrictez (*strict stubs*) . . . 8
3.6. Buenas prácticas y limitaciones . . . 8
3.7. Comparación con otras herramientas . . . 8

4. Arquitectura y componentes de Mockito 9

4.1. Conceptos previos . . . 9
4.2. Mockito explicado con una analogía . . . 10
4.3. Vista general por capas . . . 11
4.4. Módulos que se distribuyen . . . 12
4.5. Componentes de la API pública . . . 12
4.6. Componentes del núcleo interno . . . 13

4.6.1. La fachada: `Mockito` y `MockitoCore` . . . 13
4.6.2. Configuración del mock: `MockSettings` y `MockCreationSettings` . . . 13
4.6.3. La fábrica: `MockUtil.createMock` . . . 14
4.6.4. `MockMaker`: el "taller" que fabrica los mocks . . . 14
4.6.5. El instanciador: Objenesis . . . 15
4.6.6. El manejador: cadena de `MockHandler` . . . 15
4.6.7. La invocación: `Invocation` e `InvocationMatcher` . . . 15
4.6.8. El apuntador: `MockingProgress` . . . 16
4.6.9. El almacén de comodines: `ArgumentMatcherStorage` y `MatchersBinder` . . . 16
4.6.10. La libreta: `InvocationContainer` . . . 17
4.6.11. Las respuestas: `Answer` . . . 17
4.6.12. La verificación: `VerificationMode` . . . 17
4.6.13. Los mensajes: `Reporter` y limpieza de trazas . . . 18
4.7. Recorrido paso a paso: qué pasa en cada línea . . . 18
4.7.1. (A) Creación del mock . . . 19
4.7.2. (B) Stubbing: `when(inv.stockDisponible("P1")).thenReturn(5)` . . . 19
4.7.3. (C) Uso del mock desde el código bajo prueba . . . 21
4.7.4. (D) Verificación . . . 21
4.8. Ciclo de vida de una prueba con `MockitoExtension` . . . 22
4.8.1. Procesamiento de anotaciones e inyección . . . 22
4.9. Estrictez: el "control de calidad" de las pruebas . . . 22
4.10. Mocks estáticos y de constructores . . . 23
4.11. Extensibilidad: el sistema de plugins . . . 23
4.12. Demostración práctica de la arquitectura . . . 24
4.13. Diagnóstico: qué componente produce cada error . . . 25
4.14. Resumen del capítulo . . . 26

5. SRS – Especificación de Requisitos del Sistema de Ejemplo 27
5.1. Introducción . . . 27
5.1.1. Propósito . . . 27
5.1.2. Alcance . . . 27
5.1.3. Definiciones, acrónimos y abreviaturas . . . 27
5.1.4. Referencias . . . 28
5.1.5. Visión general del documento . . . 28
5.2. Descripción general . . . 28
5.2.1. Perspectiva del producto . . . 28
5.2.2. Funciones del producto . . . 28
5.2.3. Actores . . . 28
5.2.4. Restricciones . . . 29
5.2.5. Suposiciones y dependencias . . . 29

5.3. Requisitos específicos . . . 29
5.3.1. Interfaces externas . . . 29
5.3.2. Requisitos funcionales . . . 29
5.3.3. Reglas de negocio . . . 31
5.3.4. Casos de uso . . . 31
5.3.5. Requisitos no funcionales . . . 32
5.4. Observaciones de la revisión de la SRS . . . 32

6. SDD – Documento de Diseño del Sistema de Ejemplo 33
6.1. Introducción . . . 33
6.1.1. Propósito . . . 33
6.1.2. Alcance . . . 33
6.1.3. Definiciones . . . 33
6.1.4. Partes interesadas e inquietudes . . . 33
6.2. Requisitos de referencia . . . 34
6.3. Punto de vista de contexto . . . 34
6.4. Punto de vista de composición y lógico . . . 34
6.5. Punto de vista de dependencias . . . 35
6.6. Punto de vista de interacción . . . 35
6.7. Punto de vista de estados . . . 36
6.8. Punto de vista de algoritmo . . . 36
6.9. Diseño de las pruebas . . . 37
6.9.1. Matriz de trazabilidad . . . 37
6.10. Justificación de las decisiones de diseño . . . 38

7. Ejemplos prácticos de uso 39
7.1. Ciclo básico: crear, programar, ejecutar, verificar . . . 39
7.2. Anotaciones e inyección . . . 39
7.3. Argument matchers y ArgumentCaptor . . . 39
7.4. Excepciones y métodos void . . . 40
7.5. Spies (mocks parciales) . . . 40
7.6. Estilo BDD . . . 40
7.7. Orden de llamadas y métodos estáticos . . . 41
7.8. Strict stubs . . . 41

8. Cómo se diseña y se escribe un caso de prueba, paso a paso 42
8.1. El proceso general . . . 42
8.2. La ficha de un caso de prueba . . . 42
8.3. Paso 1 aplicado: ¿qué se simula y qué no? . . . 43
8.4. Paso 6 aplicado: construir una prueba de rojo a verde . . . 44
8.4.1. Versión 1: la estructura . . . 44

8.4.2. Versión 2: Arrange – Act – Assert sin programar los mocks (rojo) 44
8.4.3. Versión 3: programar solo el stock (rojo otra vez) 45
8.4.4. Versión 4: todos los stubs del camino (verde) 45
8.4.5. Versión 5: verificar el comportamiento 46
8.4.6. Versión 6: ¿qué pasa si la prueba está mal? (rojo) 46
8.4.7. Resumen: lista de control para escribir cualquier prueba 46

9. Pruebas de caja negra, paso a paso **48**
9.1. `CalculadoraDescuentos`: clases de equivalencia y valores límite 48
9.1.1. Paso 1: elemento de prueba y dependencias 48
9.1.2. Paso 2: base de prueba 48
9.1.3. Paso 3: condiciones de prueba (entradas y salidas) 48
9.1.4. Paso 4a: clases de equivalencia 48
9.1.5. Paso 4b: valores límite 49
9.1.6. Paso 4c: combinación de las dos variables 49
9.1.7. Paso 5: oráculo y tabla de casos 49
9.1.8. Paso 6: de la tabla al código, paso a paso 50
9.1.9. Paso 7: ejecución 51
9.2. `PedidoService.procesarPedido`: tabla de decisión 51
9.2.1. Pasos 1 y 2: elemento, dependencias y base de prueba 52
9.2.2. Paso 3: condiciones y acciones 52
9.2.3. Paso 4a: tabla de decisión completa 52
9.2.4. Paso 4b: simplificar la tabla 52
9.2.5. Paso 4c: valor límite de la condición C2 53
9.2.6. Paso 5: de cada regla a datos concretos y stubs 53
9.2.7. Fichas de los casos 53
9.2.8. Paso 6: código de un caso, línea por línea 54
9.2.9. Paso 7: ejecución 55

10. Pruebas de caja blanca, paso a paso **56**
10.1. `procesarPedido`: camino básico 56
10.1.1. Paso 1: numerar el código y marcar las decisiones 56
10.1.2. Paso 2: agrupar las líneas en nodos 56
10.1.3. Paso 3: dibujar el grafo y listar las aristas 57
10.1.4. Paso 4: calcular la complejidad ciclomática 58
10.1.5. Paso 5: derivar los caminos (método del camino base) 58
10.1.6. Paso 6: sensibilizar cada camino (¿qué datos lo obligan?) 58
10.1.7. Paso 7: ¿cómo se demuestra que se recorrió el camino? 59
10.1.8. Paso 8: fichas de los casos 59
10.1.9. Paso 9: código de un caso, anotado con los nodos 61
10.1.10. Paso 10: ejecutar y medir la cobertura 61

10.2. `calcularDescuento`: cobertura de decisiones 62
10.2.1. Pasos 1 y 2: código numerado y nodos 62
10.2.2. Pasos 3 y 4: aristas y complejidad 62
10.2.3. Paso 5: caminos por inversión de decisiones 62
10.3. Comparación de los dos enfoques 63

11. Resultados **64**
11.1. Ejecución de las pruebas 64
11.2. Cobertura (JaCoCo 0.8.15) 64

12. Conclusiones y recomendaciones **66**
12.1. Conclusiones 66
12.2. Recomendaciones 66

Referencias **67**

A. Estructura del proyecto y ejecución **68**

Índice de figuras

2.1. Pirámide de pruebas . . . 3

4.1. Arquitectura de Mockito por capas . . . 11
4.2. Mapa de componentes internos y sus relaciones . . . 13
4.3. Dos formas de fabricar un mock de una clase concreta . . . 15
4.4. Cadena de manejadores de cada mock . . . 15
4.5. Los comodines viajan por una pila, no dentro de los argumentos . . . 16
4.6. Secuencia de creación de un mock . . . 19
4.7. Secuencia de `when(...).thenReturn(...)` . . . 20
4.8. Secuencia de `verify(mock).metodo(...)` . . . 21
4.9. Ciclo de vida de una prueba con `MockitoExtension` . . . 22

5.1. Relación entre SRS, SDD, código y pruebas . . . 27

6.1. Diagrama de contexto. Los sistemas externos (línea discontinua) se sustituyen por mocks en las pruebas. . . . 34
6.2. Diagrama de clases del Servicio de Pedidos (interfaces en verde = puertos *mockeables*) . . . 35
6.3. Diagrama de secuencia de `procesarPedido` (pago aprobado) . . . 36
6.4. Diagrama de estados del pedido . . . 36

8.1. Proceso que seguimos para cada caso de prueba . . . 42
8.2. Criterio para decidir el tipo de doble . . . 43

9.1. Particiones, valores límite (rojo) y representantes (azul) . . . 49

10.1. Grafo de flujo de `procesarPedido` . . . 57
10.2. Grafo de `calcularDescuento` . . . 62

11.1. Ramas cubiertas de `PedidoService` según el tipo de prueba . . . 65

Índice de tablas

2.1. Niveles de prueba . . . 3
2.2. Tipos de dobles de prueba y su equivalente en Mockito . . . 4
2.3. Criterios de cobertura de caja blanca (de menor a mayor exigencia) . . . 5

3.1. Ficha técnica . . . 6
3.2. Resumen de la API de Mockito . . . 7
3.3. Mockito frente a otras herramientas de mocking . . . 8

4.1. Glosario de conceptos base . . . 9
4.2. De la analogía a los componentes reales de Mockito . . . 10
4.3. Artefactos de Mockito en Maven Central (grupo `org.mockito`) . . . 12
4.4. Componentes públicos (paquete `org.mockito`) . . . 12
4.5. Las tres implementaciones de `MockMaker` . . . 14
4.6. Respuestas por defecto (`org.mockito.Answers`) . . . 17
4.7. Modos de verificación y sus clases . . . 18
4.8. Niveles de estrictez . . . 23
4.9. Plugins disponibles (`org.mockito.plugins`) . . . 24
4.10. Qué componente demuestra cada prueba . . . 25
4.11. Errores frecuentes, componente que los detecta y causa . . . 25

5.1. Actores del sistema . . . 28
5.2. Interfaces externas (serán los puntos de sustitución por mocks) . . . 29
5.3. Reglas de negocio . . . 31
5.4. Requisitos no funcionales . . . 32
5.5. Hallazgos de la revisión de la SRS . . . 32

6.1. Partes interesadas del diseño . . . 33
6.2. Requisito → elemento de diseño . . . 34
6.3. Paquetes del sistema . . . 35
6.4. Estrategia de dobles por dependencia . . . 37
6.5. Trazabilidad SRS → diseño → casos de prueba . . . 38

8.1. Campos de la ficha de caso de prueba . . . 43
8.2. Decisión aplicada a las dependencias de `PedidoService` . . . 44

9.1. Análisis de entradas y salidas . . . 48
9.2. Clases de equivalencia derivadas . . . 49
9.3. Derivación de los valores límite . . . 49

9.4. Casos de prueba de caja negra para `CalculadoraDescuentos` . . . 50
9.5. Matriz clases × casos . . . 50
9.6. Condiciones y acciones extraídas de la SRS . . . 52
9.7. Tabla de decisión completa (8 reglas) . . . 52
9.8. Tabla de decisión simplificada (4 reglas = 4 casos) . . . 53
9.9. Condición → cómo se controla . . . 53

10.1. Decisiones encontradas en el código . . . 56
10.2. Correspondencia nodo ↔ líneas de código . . . 57
10.3. Derivación de los caminos independientes . . . 58
10.4. ¿De qué depende cada decisión? . . . 58
10.5. Valores de cada decisión y datos que los producen . . . 59
10.6. Evidencia observable de cada nodo . . . 59
10.7. Ramas cubiertas por línea de `PedidoService.java` (datos reales de JaCoCo) . . . 61
10.8. Caminos de `calcularDescuento`, obtenidos invirtiendo una decisión a la vez . . . 62
10.9. Caja negra frente a caja blanca en este proyecto . . . 63

11.1. Resumen de pruebas por categoría . . . 64
11.2. Cobertura de la capa de servicio . . . 64

Índice de códigos

3.1. Dependencias Maven (`pom.xml`) . . . 6
4.1. Prueba de referencia para el recorrido . . . 18
4.2. Cambiar globalmente el `MockMaker` mediante el sistema de plugins . . . 23
4.3. Salida de `E09` y `E10` (consola) . . . 24
6.1. Inyección por constructor en `PedidoService` . . . 35
6.2. Método `procesarPedido` con sus decisiones marcadas (D1–D6) . . . 36
7.1. `E01_PrimerMockTest` (extracto) . . . 39
7.2. `E02_AnotacionesTest` (extracto) . . . 39
7.3. `E03_ArgumentMatchersYCaptorTest` (extracto) . . . 39
7.4. `E04_ExcepcionesYVoidTest` (extracto) . . . 40
7.5. `E05_SpyTest` (extracto) . . . 40
7.6. `E06_BDDMockitoTest` (extracto) . . . 40
7.7. `E07_OrdenYStaticTest` (extracto) . . . 41
7.8. `E08_StrictStubsTest` (extracto) . . . 41
8.1. `Paso1_EstructuraTest`: preparar el escenario . . . 44
8.2. `Paso2_RojoSinStubsTest` . . . 44
8.3. Salida real de la ejecución . . . 45
8.4. `Paso3_RojoStubIncompletoTest`: se añade una línea . . . 45
8.5. Salida real de la ejecución . . . 45
8.6. `Paso4_VerdeTest` . . . 45
8.7. `Paso5_VerificarComportamientoTest`: se añaden 3 líneas . . . 46
8.8. `Paso6_RojoVerificacionTest` . . . 46
8.9. Salida real de la ejecución . . . 46
9.1. Un caso aislado . . . 50
9.2. Clases válidas y límites (`CalculadoraDescuentosCajaNegraTest`) . . . 51
9.3. Clase inválida CE1 (CD-13 y CD-14) . . . 51
9.4. Salida de la ejecución (extracto; los ID entre paréntesis son anotaciones nuestras) . . . 51
9.5. Regla R4 comentada paso a paso . . . 54
9.6. Salida de la ejecución (ordenada) . . . 55
10.1. Camino C5: cada línea de la prueba se relaciona con un nodo del grafo . . . 61
10.2. Salida de la ejecución (ordenada) . . . 61
11.1. Ejecución con `./run.sh cobertura` (resumen) . . . 64
A.1. Estructura de `ejemplos-mockito/` . . . 68
A.2. Comandos de ejecución . . . 68

Capítulo 1

Introducción

1.1 Antecedentes

Un sistema real casi nunca trabaja solo. Un servicio de pedidos consulta el inventario en una base de datos, cobra a través de un banco y envía correos de confirmación. Probar ese servicio junto con todas sus dependencias reales trae varios problemas:

- **Lentitud**: cada prueba abre conexiones de red o de base de datos.
- **Fragilidad**: si el banco está caído, la prueba falla aunque nuestro código esté bien.
- **Efectos secundarios**: no queremos cobrar dinero real ni enviar correos a clientes reales.
- **Escenarios difíciles de reproducir**: ¿cómo provocar a voluntad un *"pago rechazado por fondos insuficientes"* o una base de datos caída?

La solución son los **dobles de prueba**: objetos que se hacen pasar por las dependencias reales y que el tester controla por completo. Mockito es la herramienta estándar de facto para crearlos en Java.

1.2 Objetivos

1.2.1 Objetivo general

Explicar el funcionamiento y el uso del framework Mockito, y aplicarlo en pruebas de caja blanca y caja negra sobre un sistema de ejemplo documentado con un SDD.

1.2.2 Objetivos específicos

1. Describir los conceptos de prueba unitaria y dobles de prueba, y las técnicas de caja negra y caja blanca.
2. Explicar la arquitectura de Mockito y cada uno de sus componentes (fachada, *mock makers*, manejadores, registro de invocaciones, matchers, verificación, plugins) y su API principal (*stubbing*, verificación, *matchers*, *captors*, *spies*, BDD, métodos estáticos).
3. Elaborar la especificación de requisitos (SRS, IEEE 830 / ISO/IEC/IEEE 29148) y el documento de diseño (SDD, IEEE 1016) de un sistema de ejemplo diseñado para ser *testeable*.
4. Implementar ejemplos prácticos ejecutables que muestren cada característica de Mockito.
5. Explicar paso a paso cómo se diseñan los casos de prueba de caja negra (clases de equivalencia, valores límite, tabla de decisión) y de caja blanca (grafo de flujo, camino básico, cobertura de ramas), y cómo se escribe cada prueba.
6. Medir la cobertura y relacionar los resultados con la auditoría de sistemas.

1.3 Justificación

Las normas y marcos de referencia de auditoría de TI, como COBIT 2019 en su dominio BAI03 (*Gestionar la identificación y construcción de soluciones*), incluido el diseño y la ejecución de pruebas, e ISO/IEC/IEEE 29119 (*Software Testing*), piden que el software se pruebe de forma planificada y documentada, y que existan evidencias verificables de esas pruebas. Las pruebas unitarias automatizadas con Mockito cumplen esas condiciones: se ejecutan en milisegundos, se repiten sin intervención humana, se conectan a los requisitos y dejan métricas objetivas como el número de casos o el porcentaje de cobertura. Conocer la herramienta permite al auditor evaluar si la estrategia de pruebas de un equipo de desarrollo es suficiente.

1.4 Alcance

El trabajo abarca pruebas **unitarias** en Java con JUnit 6 y Mockito 5. No incluye pruebas de integración, de sistema ni de rendimiento, aunque se mencionan como contexto. El sistema de ejemplo es didáctico: sus dependencias externas (base de datos, banco, correo) solo existen como interfaces y en las pruebas se sustituyen por mocks.

Capítulo 2

Marco teórico

2.1 Pruebas de software

La prueba de software es el proceso de ejecutar un programa con la intención de encontrar defectos y de obtener evidencia de que cumple sus requisitos. Las pruebas se organizan por **niveles**:

Tabla 2.1: Niveles de prueba

Nivel	Qué se prueba	Herramientas típicas
Unitaria	Una clase o método de forma aislada	JUnit, **Mockito**
Integración	La colaboración entre componentes reales (p. ej., servicio + base de datos)	Testcontainers, Spring Test
Sistema	El sistema completo contra los requisitos	Selenium, Postman
Aceptación	Que el sistema satisfaga al usuario o al negocio	Cucumber, pruebas manuales

La *pirámide de pruebas* (Mike Cohn) recomienda tener muchas pruebas unitarias, rápidas y baratas, y pocas pruebas de extremo a extremo, lentas y costosas. Mockito es lo que hace viable la base de esa pirámide.

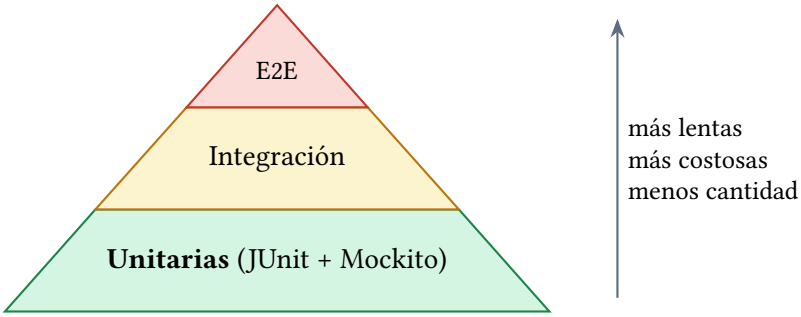


Figura 2.1: Pirámide de pruebas

2.1.1 Características de una buena prueba unitaria (FIRST)

Fast Rápida: milisegundos, sin red ni disco.

Independent Independiente: no depende del orden ni de otras pruebas.

Repeatable Repetible: da el mismo resultado en cualquier máquina y en cualquier momento.

Self-validating Autoverificable: aprueba o falla sola, sin inspección manual.

Timely Oportuna: se escribe junto con el código (o antes, en TDD).

Sin dobles de prueba es prácticamente imposible cumplir F, I y R cuando la unidad tiene dependencias externas.

2.2 Dobles de prueba

Gerard Meszaros (*xUnit Test Patterns*, 2007) acuñó el término *test double*, por analogía con el doble de riesgo del cine, y clasificó cinco tipos:

Tabla 2.2: Tipos de dobles de prueba y su equivalente en Mockito

Tipo	Descripción	En Mockito
Dummy	Objeto que solo se pasa para completar parámetros; nunca se usa.	`mock(X.class)` sin configurar
Stub	Devuelve respuestas predefinidas a las llamadas.	`when(...).thenReturn(...)`
Spy	Objeto real que además registra cómo fue usado.	`spy(obj)`, `@Spy`
Mock	Objeto con expectativas sobre las llamadas que debe recibir; se verifica.	`verify(mock)...`
Fake	Implementación funcional pero simplificada (p. ej., base de datos en memoria).	No aplica (se programa a mano)

Idea clave

En Mockito, un mismo objeto creado con `mock()` puede actuar como *stub* (cuando se le programan respuestas) y como *mock* (cuando se verifican sus llamadas). Por eso, en el habla cotidiana se le llama "mock" a todo.

2.2.1 Verificación de estado frente a verificación de comportamiento

- **Verificación de estado**: se comprueba el *resultado*, por ejemplo `assertEquals(PAGADO, pedido.getEstado())`.
- **Verificación de comportamiento**: se comprueba la *interacción*, por ejemplo `verify(pasarela).cobrar("a@b.com", 450.0)`.

Mockito permite ambas. Veremos que la primera encaja con la **caja negra** y la segunda con la **caja blanca**.

2.3 Pruebas de caja negra

En las pruebas de caja negra (funcionales o basadas en la especificación), los casos de prueba se diseñan a partir de los **requisitos**, sin mirar el código fuente. Solo interesa la relación entre entradas y salidas.

Partición en clases de equivalencia. El dominio de cada entrada se divide en clases cuyos elementos el sistema debe tratar igual. Se elige un representante de cada clase válida y de cada clase inválida.

Análisis de valores límite. Los defectos se concentran en los bordes de las clases (el típico error de < por <=). Por eso se prueban los valores exactos del límite y los inmediatamente adyacentes.

Tabla de decisión. Cuando la salida depende de combinaciones de condiciones, se tabulan todas las combinaciones (reglas) y la acción esperada para cada una.

Transición de estados. Se prueban los estados de una entidad y los eventos que provocan sus transiciones.

2.4 Pruebas de caja blanca

En las pruebas de caja blanca (estructurales), los casos se diseñan a partir de la **estructura interna del código**: sentencias, decisiones, condiciones y caminos.

Tabla 2.3: Criterios de cobertura de caja blanca (de menor a mayor exigencia)

Criterio	Exige que…
Sentencias	cada instrucción se ejecute al menos una vez.
Decisiones (ramas)	cada decisión (`if`, `while`, `for`) tome los valores verdadero y falso.
Condiciones	cada condición atómica de una decisión compuesta tome ambos valores.
Decisión/condición	se cumplan a la vez los dos criterios anteriores.
Camino básico	se ejecute un conjunto de caminos linealmente independientes ($V(G)$ caminos).

2.4.1 Complejidad ciclomática de McCabe

Thomas McCabe (1976) propuso representar un módulo como un **grafo de flujo** G y calcular su complejidad ciclomática $V(G)$, que equivale al número de caminos linealmente independientes y, por lo tanto, al **número mínimo de casos de prueba** del criterio de camino básico. Se calcula de tres formas equivalentes:

$$V(G) = E - N + 2 \qquad \text{(aristas} - \text{nodos} + 2\text{)}$$
$$V(G) = P + 1 \qquad \text{(nodos predicado o decisiones simples} + 1\text{)}$$
$$V(G) = R \qquad \text{(número de regiones del grafo planar)}$$

2.4.2 ¿Qué aporta Mockito a la caja blanca?

Para recorrer un camino concreto hay que forzar las decisiones del código. Muchas decisiones dependen de lo que devuelven las dependencias, por ejemplo *"si el banco aprueba el pago…"*. Con Mockito, un *stub* fija ese valor y hace que el programa entre en la rama deseada. Después, `verify()` confirma qué instrucciones internas se ejecutaron y cuáles no. Sin mocks, muchas ramas serían inalcanzables en una prueba unitaria.

2.5 Las pruebas en la auditoría de sistemas

Para el auditor de sistemas, las pruebas automatizadas son **evidencia** de que los controles de desarrollo funcionan:

- **Trazabilidad**: cada requisito debe tener al menos una prueba asociada (matriz de trazabilidad).
- **Repetibilidad**: el auditor puede volver a ejecutar las pruebas y obtener el mismo resultado.
- **Medición objetiva**: la cobertura de sentencias y de ramas (JaCoCo) cuantifica qué parte del código fue probada.
- **Control de cambios**: en integración continua, ningún cambio llega a producción si rompe una prueba.
- **Segregación**: el uso de mocks garantiza que las pruebas no tocan datos de producción, lo que protege la confidencialidad y la integridad.

Capítulo 3

Mockito

3.1 ¿Qué es Mockito?

Mockito es un framework de código abierto (licencia MIT) para crear dobles de prueba en Java y en otros lenguajes de la JVM, como Kotlin o Scala. Lo creó **Szczepan Faber** en 2007-2008 a partir del código de EasyMock. Buscaba una API más simple y legible, basada en el patrón *arrange–act–assert*, en la que primero se ejecuta el código y después se verifica (en lugar de declarar expectativas antes, como en EasyMock). Hoy es la biblioteca de mocking más usada en Java y viene incluida por defecto en `spring-boot-starter-test`.

Tabla 3.1: Ficha técnica

Sitio oficial	`https://site.mockito.org`
Repositorio	`https://github.com/mockito/mockito`
Versión usada	5.24.0 (Maven Central)
Requisito	Java 11 o superior (Mockito 5)
Dependencias	Byte Buddy (generación de código), Objenesis (instanciación)
Integración	JUnit 5/6 (`mockito-junit-jupiter`), JUnit 4, TestNG, Spring
Licencia	MIT

3.2 ¿Cómo funciona? Visión rápida

Mockito fabrica en tiempo de ejecución objetos que se hacen pasar por el tipo pedido, pero cuyos métodos no ejecutan el código original: cada llamada se desvía a un manejador interno que la **registra** y **responde** según lo programado. `when()` programa respuestas usando "la última llamada hecha a un mock" y `verify()` revisa el registro de llamadas. El capítulo 4 explica en detalle la arquitectura y cada uno de sus componentes. De ese diseño se derivan tres reglas prácticas:

⚠ Reglas que se derivan del funcionamiento interno

- Si un argumento usa un *matcher*, **todos** deben usarlos: `cobrar(eq("a"), anyDouble())`, y no `cobrar("a", anyDouble())`.
- Con parámetros primitivos hay que usar la variante primitiva (`anyDouble()`, `doubleThat()`). `any()` o `argThat()` devuelven `null` y producen un `NullPointerException` al convertirse a `double`. Nos pasó durante el desarrollo de los ejemplos (sección 7.3).
- Los métodos `void` no pueden ir dentro de `when(...)` porque no devuelven nada; para ellos se usa la familia `doThrow()/doAnswer()/doNothing().when(mock).metodo()`.

3.3 Instalación

```
<dependency>
  <groupId>org.mockito</groupId>
```

```
    <artifactId>mockito-core</artifactId>
    <version>5.24.0</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-junit-jupiter</artifactId>
    <version>5.24.0</version>
    <scope>test</scope>
  </dependency>
```

Código 3.1: Dependencias Maven (`pom.xml`)

En Gradle: `testImplementation("org.mockito:mockito-core:5.24.0")`.

JDK 21 o superior

Las JDK modernas desaconsejan que un agente se cargue dinámicamente durante la ejecución (JEP 451) y muestran un aviso. La recomendación oficial es registrar Mockito como agente al arrancar la JVM: `-javaagent:mockito-core-5.24.0.jar`. En el proyecto de ejemplo esto se configura en el `maven-surefire-plugin` y en el script `run.sh`.

3.4 API principal

Tabla 3.2: Resumen de la API de Mockito

API	Uso
Creación	
`mock(Clase.class)` / `@Mock`	Crea un mock.
`spy(objeto)` / `@Spy`	Envuelve un objeto real (mock parcial).
`@InjectMocks`	Crea el objeto bajo prueba e inyecta los mocks (constructor, setter o campo).
`@Captor`	Crea un `ArgumentCaptor`.
`mockStatic(Clase.class)`	Simula métodos estáticos dentro de un `try`.
Stubbing (programar respuestas)	
`when(m.f()).thenReturn(v)`	Devuelve `v`; con varios valores, respuestas consecutivas.
`thenThrow(ex)`	Lanza una excepción.
`thenAnswer(inv -> ...)`	Respuesta calculada con los argumentos recibidos.
`thenCallRealMethod()`	Ejecuta el método real.
`doReturn/doThrow/doAnswer/doNothing().when(m).f()`	Para métodos `void` y para spies.
`given(...).willReturn(...)`	Equivalente BDD de `when`.
Verificación	
`verify(m).f()`	Se llamó exactamente una vez.
`verify(m, times(n) | never() | atLeast(n) | atMost(n))`	Cantidad de llamadas.
`inOrder(m1, m2).verify(...)`	Orden de las llamadas.
`verifyNoInteractions(m)`	El mock no se usó en absoluto.
`verifyNoMoreInteractions(m)`	No hubo llamadas sin verificar.
`then(m).should().f()`	Equivalente BDD de `verify`.

API	Uso
Argument matchers	
`any()`, `any(Clase.class)`, `anyString()`, `anyInt()`, `anyDouble()`	Cualquier valor del tipo.
`eq(v)`, `isNull()`, `notNull()`	Igualdad o nulidad.
`contains()`, `startsWith()`, `matches(regex)`	Cadenas.
`argThat(lambda)`, `intThat()`, `doubleThat()`	Condición personalizada.

3.5 Estrictez (*strict stubs*)

Con `MockitoExtension`, Mockito trabaja por defecto en modo `STRICT_STUBS`: si una prueba programa un *stub* que nunca se usa, falla con `UnnecessaryStubbingException`. Así se detectan pruebas mal escritas o código muerto en las pruebas. Cuando un stub no usado es intencional, se marca con `lenient()`.

3.6 Buenas prácticas y limitaciones

- **Diseñar para la testabilidad**: recibir las dependencias por constructor (inyección de dependencias) y programar contra interfaces.
- **No simular lo que no es tuyo** (*don't mock what you don't own*): es preferible envolver las bibliotecas de terceros en una interfaz propia y simular esa interfaz.
- **No simular objetos de valor** (*records*, DTO, `String`, `List`): basta con crear instancias reales.
- **No abusar de `verify()`**: verificar cada detalle acopla la prueba a la implementación y la vuelve frágil ante refactorizaciones.
- **Un comportamiento por prueba** y nombres descriptivos (`@DisplayName`).
- **Limitaciones**: Mockito no simula `equals()` ni `hashCode()`, y tampoco métodos privados. Los mocks estáticos solo valen en el hilo actual y dentro de su bloque. Un mock no garantiza que la dependencia real se comporte igual; para eso están las pruebas de integración.

3.7 Comparación con otras herramientas

Tabla 3.3: Mockito frente a otras herramientas de mocking

Herramienta	Lenguaje	Observaciones
Mockito	Java/JVM	API simple; verificación posterior; estándar de facto.
EasyMock	Java	Modelo *record–replay*: las expectativas se declaran antes.
JMockit / PowerMock	Java	Simulaban estáticos, constructores y `final`; hoy están en desuso porque Mockito inline ya cubre esos casos.
MockK	Kotlin	Pensado para Kotlin (corrutinas, objetos).
unittest.mock	Python	Incluido en la biblioteca estándar.
Moq / NSubstitute	C#	Equivalentes en .NET.
Jest (`jest.fn()`)	JavaScript	*Mocks* integrados en el runner.

Capítulo 4

Arquitectura y componentes de Mockito

Este capítulo explica **cómo está construido Mockito por dentro**: de qué piezas se compone, qué hace cada una y cómo colaboran cuando se escribe una línea como esta:

```
when(inventario.stockDisponible("P1")).thenReturn(5);
```

Empieza desde cero, con conceptos básicos y una analogía, y avanza hasta el nivel de las clases reales de la biblioteca.

¿De dónde sale esta información?

Los nombres de clases, paquetes y el orden de las operaciones de este capítulo se verificaron sobre el propio `mockito-core-5.24.0.jar`: se listó su contenido y se inspeccionó el bytecode con `javap`. Además, los ejemplos `E09_ArquitecturaInternaTest` y `E10_ComponentesExtensiblesTest` del proyecto *demuestran en ejecución* cada afirmación importante (sección 4.12).

4.1 Conceptos previos

Para entender la arquitectura hacen falta algunos conceptos de Java y de la máquina virtual. Si ya se conocen, se puede saltar esta sección.

Tabla 4.1: Glosario de conceptos base

Concepto	Explicación sencilla
Clase	El "molde" o plano con el que se fabrican objetos. Ej.: `PasarelaPago`.
Objeto (instancia)	Una "pieza" concreta fabricada con el molde. Ej.: la pasarela que usa el sistema.
Interfaz	Un contrato: dice *qué* métodos debe tener un objeto, pero no *cómo* funcionan. Ej.: `PasarelaPago` solo declara `cobrar(email, monto)`.
Método	Una acción que un objeto sabe hacer. Llamar a un método = *invocarlo*.
JVM	*Java Virtual Machine*: el programa que ejecuta el código Java.
Bytecode	El "idioma" que entiende la JVM. El compilador traduce el `.java` a bytecode (`.class`). Quien modifica el bytecode cambia lo que hace la clase *sin tocar el código fuente*.
ClassLoader	El componente de la JVM que carga las clases en memoria cuando se necesitan.
Reflexión	La capacidad de un programa de examinarse a sí mismo en ejecución: preguntar qué métodos tiene una clase, invocarlos por nombre, leer campos privados, etc.
Proxy dinámico	Un objeto generado en ejecución que implementa una interfaz y desvía *todas* las llamadas a un único "manejador".
Agente Java	Un programa que se engancha a la JVM (opción `-javaagent` o conexión dinámica) y obtiene la *Instrumentation API*, que permite **reescribir el bytecode de clases ya cargadas**.
Hilo (*thread*)	Una línea de ejecución. Una prueba corre, normalmente, en un solo hilo.
`ThreadLocal`	Una "caja" de datos en la que cada hilo ve su propio contenido. Mockito guarda en una de ellas su estado de trabajo.

Concepto	Explicación sencilla
Generación de código	Crear clases nuevas en tiempo de ejecución (sin escribirlas a mano). Mockito usa la biblioteca Byte Buddy para eso.

4.2 Mockito explicado con una analogía

Imaginemos que una prueba es el **ensayo de una obra de teatro**:

- La **obra** es la prueba unitaria.
- El **actor principal** es la clase que queremos evaluar (`PedidoService`). Es el único actor real.
- Los **actores secundarios** (banco, base de datos, correo) no vienen al ensayo; en su lugar se usan **marionetas** (los mocks).
- Un **taller** fabrica las marionetas con la forma exacta de cada actor, para que el actor principal no note la diferencia.
- El **director** (quien escribe la prueba) entrega a cada marioneta un **guion**: "si te preguntan por el stock de P1, responde 5".
- Cada marioneta lleva una **grabadora** que registra todo lo que le dijeron.
- Al final, el director **revisa la grabación**: "¿le pidieron al banco cobrar 450 Bs exactamente una vez?".
- Un **apuntador** entre bastidores recuerda cuál fue la última frase dicha a una marioneta y qué está haciendo el director en ese momento (escribiendo un guion o revisando una grabación).

Tabla 4.2: De la analogía a los componentes reales de Mockito

En la analogía	Qué hace	Componente real
Marioneta	Se hace pasar por el actor	El *mock*
Taller	Fabrica la marioneta con la forma correcta	`MockMaker` + Byte Buddy + Objenesis
Ficha técnica del pedido al taller	Tipo, nombre, respuesta por defecto…	`MockSettings` / `MockCreationSettings`
"Cerebro" de la marioneta	Recibe cada frase y decide qué contestar	`MockHandler`
Grabadora + libreta de guiones	Guarda las llamadas y las respuestas programadas	`InvocationContainer`
Guion	"Si te dicen X, responde Y"	*Stub*: `when().thenReturn()` y `Answer`
Apuntador	Recuerda la última llamada y el modo actual	`MockingProgress` (por hilo)
Comodines del guion	"Cualquier producto", "cualquier monto"	`ArgumentMatcher` (`any()`, `eq()`…)
Revisión de la grabación	Comprobar qué se dijo y cuántas veces	`verify()` + `VerificationMode`
Crítico que señala errores	Explica claramente qué salió mal	`Reporter` y excepciones de Mockito
Jefe de escena	Prepara todo antes del ensayo y revisa al final	`MockitoExtension` / `MockitoSession`

4.3 Vista general por capas

Mockito se organiza en capas. Cada capa solo conoce a la de abajo, lo que permite cambiar piezas (por ejemplo, la forma de fabricar mocks) sin afectar al usuario.

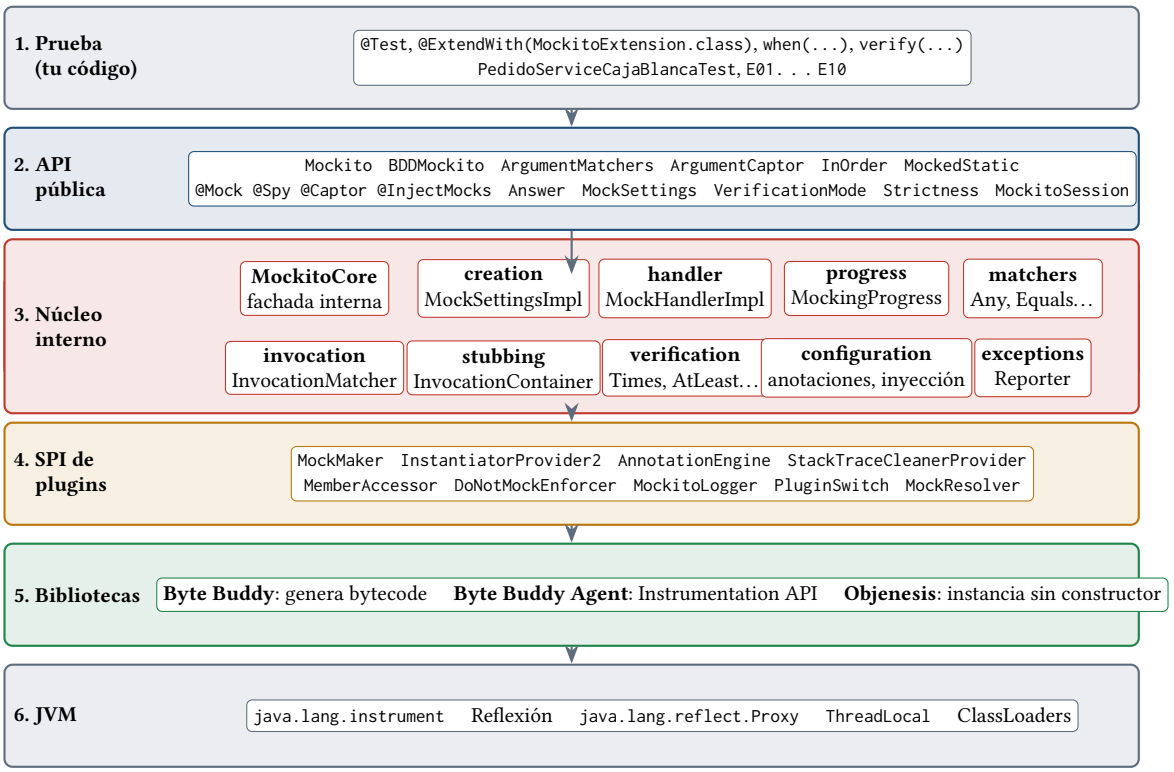


Figura 4.1: Arquitectura de Mockito por capas

1. Prueba. El código que escribe el tester. Solo usa la capa 2.

2. API pública (paquete `org.mockito`). Es lo único que se debería usar. Tiene garantía de compatibilidad entre versiones.

3. Núcleo interno (paquete `org.mockito.internal`). Implementa toda la lógica: crear mocks, interceptar llamadas, guardar stubs, verificar y reportar errores. Puede cambiar entre versiones y no debe usarse directamente.

4. SPI de plugins (paquete `org.mockito.plugins`). *Service Provider Interface*: puntos de enchufe con los que se puede reemplazar una pieza (por ejemplo, la fábrica de mocks) sin modificar Mockito.

5. Bibliotecas externas. Byte Buddy (creada por Rafael Winterhalter, también mantenedor de Mockito) y Objenesis.

6. JVM. Las capacidades de la plataforma sobre las que todo se apoya.

4.4 Módulos que se distribuyen

Tabla 4.3: Artefactos de Mockito en Maven Central (grupo `org.mockito`)

Artefacto	Para qué sirve
`mockito-core`	El framework completo. Es el único imprescindible.
`mockito-junit-jupiter`	Integración con JUnit 5/6: `MockitoExtension`, `@MockitoSettings` y la inyección de mocks como parámetros de métodos de prueba.
`mockito-subclass`	Cambia el *mock maker* por defecto a *subclass* (para entornos donde no se puede usar un agente).
`mockito-proxy`	Cambia el *mock maker* por defecto a *proxy* (solo interfaces, sin generación de código).
`mockito-android`	Variante para Android (genera código compatible con la máquina virtual de Android).
`mockito-errorprone`	Reglas de análisis estático para detectar malos usos de Mockito al compilar.
`mockito-bom`	*Bill of materials*: fija las versiones de todos los módulos a la vez.
`mockito-inline`	**Obsoleto**: su función pasó a ser la predeterminada en Mockito 5 (última versión: 5.2.0).

4.5 Componentes de la API pública

Tabla 4.4: Componentes públicos (paquete `org.mockito`)

Componente	Responsabilidad
`Mockito`	Punto de entrada. Métodos estáticos: `mock`, `spy`, `when`, `verify`, `doReturn...`, `mockStatic`, `mockConstruction`, `mockingDetails`. Hereda de `ArgumentMatchers`.
`BDDMockito`	Alias con estilo *Given/When/Then*: `given()`, `willReturn()`, `then().should()`.
`ArgumentMatchers`	Comodines para argumentos: `any`, `eq`, `anyString`, `argThat...`
`ArgumentMatcher<T>`	Interfaz funcional `boolean matches(T arg)` para crear comodines propios.
`ArgumentCaptor<T>`	Captura los argumentos reales con que se llamó a un mock.
`MockSettings`	Configuración avanzada al crear un mock: nombre, respuesta por defecto, interfaces extra, `mockMaker`, `useConstructor`, `invocationListeners`, `strictness...`
`Answer<T>`	Interfaz que decide la respuesta de un mock a partir de la invocación: `T answer(InvocationOnMock inv)`.
`OngoingStubbing<T>`	Lo que devuelve `when()`: permite encadenar `thenReturn`, `thenThrow`, `thenAnswer`.
`Stubber`	Lo que devuelven `doReturn/doThrow/doAnswer/doNothing`; se completa con `.when(mock)`.
`VerificationMode`	Cómo verificar: `times(n)`, `never()`, `atLeast(n)`, `atMost(n)`, `only()`, `timeout(ms)`.
`InOrder`	Verifica el orden de las llamadas entre uno o varios mocks.
`MockedStatic<T>`, `MockedConstruction<T>`	Controlan la simulación de métodos estáticos y de constructores; se cierran con `close()` (*try-with-resources*).

Componente	Responsabilidad
`MockingDetails`	"Radiografía" de un mock: invocaciones, stubs, configuración, manejador.
`@Mock`, `@Spy`, `@Captor`, `@InjectMocks`	Anotaciones que evitan escribir la creación a mano.
`MockMakers`	Constantes para elegir la fábrica por mock: `INLINE`, `SUBCLASS`, `PROXY`.
`Strictness`	Nivel de exigencia: `LENIENT`, `WARN`, `STRICT_STUBS`.
`MockitoSession`	Ciclo de vida de una prueba: inicia mocks y al final valida el uso y la estrictez.
`Answers`	Respuestas por defecto predefinidas (`RETURNS_DEFAULTS`, `RETURNS_DEEP_STUBS`...).

4.6 Componentes del núcleo interno

Esta sección recorre, una por una, las piezas del paquete `org.mockito.internal`. La figura 4.2 muestra cómo se relacionan.

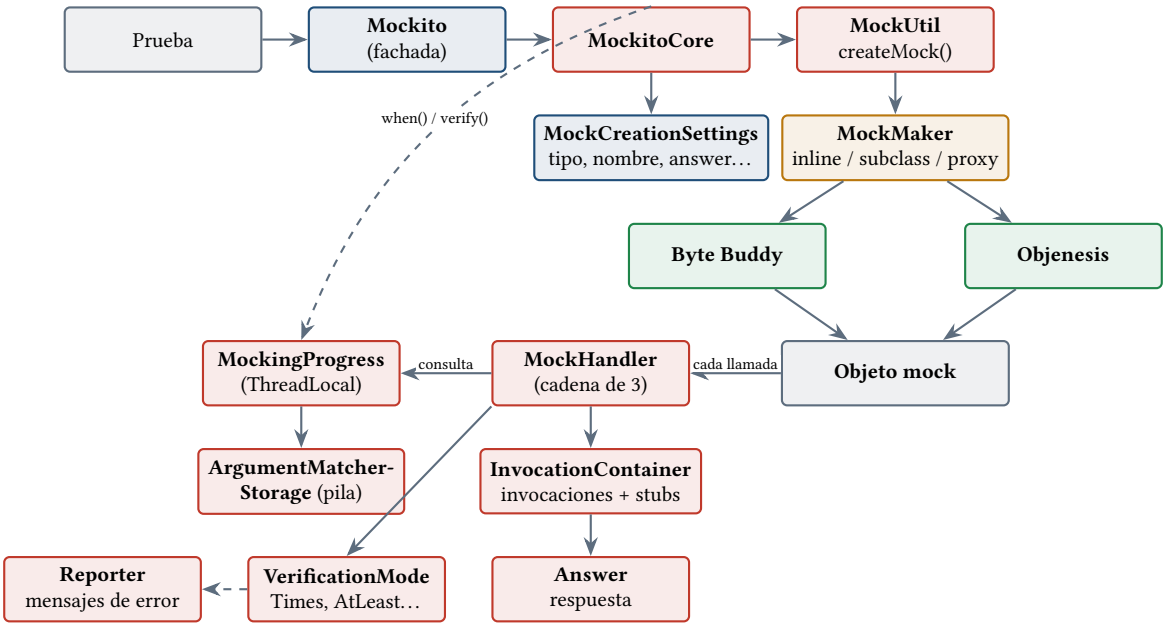


Figura 4.2: Mapa de componentes internos y sus relaciones

4.6.1 La fachada: `Mockito` y `MockitoCore`

`Mockito` es una clase con métodos estáticos pensada para leerse como lenguaje natural. No contiene lógica: cada método delega en una única instancia de `org.mockito.internal.MockitoCore`. Este patrón se llama **fachada** (*facade*): una puerta de entrada simple que esconde un sistema complejo. Así, el equipo de Mockito puede reorganizar el interior sin romper las pruebas de los usuarios.

4.6.2 Configuración del mock: `MockSettings` y `MockCreationSettings`

Cuando se escribe `mock(PasarelaPago.class)`, internamente se arma una ficha de configuración (`MockSettingsImpl`) que luego se valida y se congela como `MockCreationSettings`. Contiene:

- el **tipo** a simular (`typeToMock`) y las interfaces adicionales;

- el **nombre** del mock (aparece en los mensajes de error);
- la **respuesta por defecto** (`defaultAnswer`), por ejemplo `RETURNS_DEFAULTS`;
- la **instancia espiada**, si es un `spy`;
- si debe usar un **constructor real** (`useConstructor()`);
- la **estrictez**, los **listeners** de invocaciones y el `mockMaker` elegido;
- si es serializable o *stub-only* (sin registro de invocaciones, para ahorrar memoria).

4.6.3 La fábrica: `MockUtil.createMock`

`MockUtil` coordina la creación en tres pasos:

1. Crea el **manejador** con `MockHandlerFactory.createMockHandler(settings)`.
2. Pide al **`MockMaker`** que fabrique el objeto mock y lo conecte con ese manejador.
3. Si es un `spy`, copia el estado del objeto real al mock con `LenientCopyTool` (campo por campo).

4.6.4 `MockMaker`: el "taller" que fabrica los mocks

`MockMaker` es un *plugin* (interfaz del paquete `org.mockito.plugins`) con tres implementaciones. Es la pieza que más cambió en la historia de Mockito.

Tabla 4.5: Las tres implementaciones de `MockMaker`

	Inline (por defecto)	**Subclass**	**Proxy**
Clase	`InlineByteBuddyMockMaker`	`SubclassByteBuddyMockMaker`	`ProxyMockMaker`
Técnica	Reescribe el bytecode de la clase original (agente + Instrumentation API)	Genera una subclase `Tipo$MockitoMock$xxxx` con Byte Buddy	`java.lang.reflect.Proxy` de la JDK
Clases `final`	Sí	No	No
Métodos estáticos	Sí (`mockStatic`)	No	No
Constructores	Sí (`mockConstruction`)	No	No
Solo interfaces	No	No	Sí
Necesita agente	Sí	No	No
`getClass()` del mock	La misma clase	La subclase generada	`jdk.proxy2.$ProxyNN`

Cómo trabaja el *mock maker* inline.

1. Al primer uso, **Byte Buddy Agent** se conecta a la JVM y obtiene la *Instrumentation API* (de ahí la recomendación de pasar `-javaagent` en JDK 21 o superior).
2. Mockito inyecta en el *classloader* de arranque una pequeña clase puente, `MockMethodDispatcher`, visible desde cualquier clase del sistema.
3. `InlineBytecodeGenerator` **retransforma** la clase a simular (y sus superclases): al inicio de *cada* método inserta un fragmento de código (*advice*, clase `MockMethodAdvice`) que pregunta: "¿el objeto `this` es un mock registrado?".
4. Mockito mantiene un mapa débil (*weak map*) *objeto* → *interceptor* con todos los mocks vivos. Si `this` está en el mapa, la llamada se desvía al `MockHandler` y el código original **no se ejecuta**. Si no está, el método corre normalmente.

Por eso, con el *mock maker* inline, un mock de `CalculadoraDescuentos` **es** un `CalculadoraDescuentos` (misma clase), y aun así los objetos reales de esa clase siguen funcionando con normalidad. Lo comprueba `E09_ArquitecturaInternaTest`. Si se simula una *interfaz*, no hay código que reescribir, así que se genera una clase que la implementa (`InventarioRepository$MockitoMock$YWsjgKQa`).



Figura 4.3: Dos formas de fabricar un mock de una clase concreta

4.6.5 El instanciador: Objenesis

Una vez que existe la clase del mock, hay que crear un objeto. Llamar al constructor real sería peligroso: podría abrir conexiones o lanzar excepciones. Por eso Mockito usa **Objenesis** (`ObjenesisInstantiator`), una biblioteca que crea instancias **sin ejecutar ningún constructor**, con mecanismos internos de la JVM. Si se pide explícitamente `withSettings().useConstructor(...)`, se usa `ConstructorInstantiator`. La elección la hace el plugin `InstantiatorProvider2`.

4.6.6 El manejador: cadena de `MockHandler`

Cada mock tiene su propio manejador, formado por tres piezas encadenadas (patrón *decorator*):

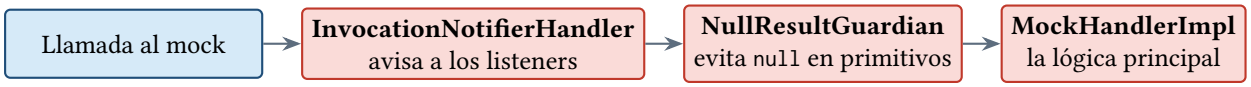


Figura 4.4: Cadena de manejadores de cada mock

- **`InvocationNotifierHandler`** informa de cada llamada a los `InvocationListener` registrados (útil para depurar; ver `E10`).
- **`NullResultGuardian`** garantiza que un método que devuelve un primitivo (`int`, `double`…) nunca devuelva `null`; en ese caso entrega `0` o `false` y evita un `NullPointerException`.
- **`MockHandlerImpl`** decide si la llamada es parte de un *stubbing*, de una *verificación* o una llamada normal, y actúa en consecuencia. Su algoritmo se detalla en la sección 4.7.

4.6.7 La invocación: `Invocation` e `InvocationMatcher`

Cada llamada a un mock se empaqueta en un objeto `InterceptedInvocation` que guarda: el mock, el método, los argumentos, una referencia al *método real* (para los spies), la *ubicación* en el código fuente (archivo y línea, para los mensajes de error) y un *número de secuencia* global (para `InOrder`).

Un `InvocationMatcher` es una invocación "con comodines": la invocación más una lista de `ArgumentMatcher`, uno por argumento. Es la forma en que Mockito representa "`stockDisponible(cualquier String)`". Sirve para buscar stubs y para verificar.

4.6.8 El apuntador: `MockingProgress`

`MockingProgress` es la **memoria de trabajo** de Mockito. Hay una por hilo (`ThreadSafeMockingProgress` la guarda en un `ThreadLocal`), por lo que las pruebas que corren en paralelo en distintos hilos no se mezclan. Guarda:

- el **`OngoingStubbing` pendiente**: la *última* invocación hecha a *cualquier* mock, lista para que `when()` la tome;
- el **modo de verificación pendiente**: si se acaba de llamar a `verify(mock)`, la próxima llamada a ese mock es una verificación, no un uso;
- si hay un **stubbing en curso** (para detectar `when(...)` sin `thenReturn`);
- el **almacén de matchers** (`ArgumentMatcherStorage`);
- los *listeners* del framework.

⚠ Consecuencia práctica comprobada

Como `when()` usa "la última invocación a un mock del hilo", el código `inventario.stockDisponible("P1"); when(listaReal.size()).thenReturn(10);` **programa el mock `inventario`**, no la lista real. `E09` lo demuestra: después, `inventario.stockDisponible("P1")` devuelve `10`. Si no hay ninguna invocación pendiente, `when()` lanza `MissingMethodInvocationException`.

4.6.9 El almacén de comodines: `ArgumentMatcherStorage` y `MatchersBinder`

Los comodines no se pasan "dentro" del argumento. Cuando Java evalúa `anyString()`, este método:

1. crea un objeto `Any` (un `ArgumentMatcher`);
2. lo **apila** en una pila (`java.util.Stack`) de `ArgumentMatcherStorageImpl`, en el `MockingProgress` del hilo;
3. devuelve un valor ficticio del tipo correcto (`null`, `""`, `0`, `false`) para que el código compile y se pueda llamar al mock.

Cuando el mock recibe la llamada, `MatchersBinder` **vacía la pila** y la asocia a los argumentos:

- si la pila está vacía, cada argumento real se envuelve en un comparador `Equals` (lo hace `ArgumentsProcessor`);
- si la pila tiene tantos comodines como argumentos, se usan esos;
- si no coinciden, se lanza `InvalidUseOfMatchersException` ("*2 matchers expected, 1 recorded*").

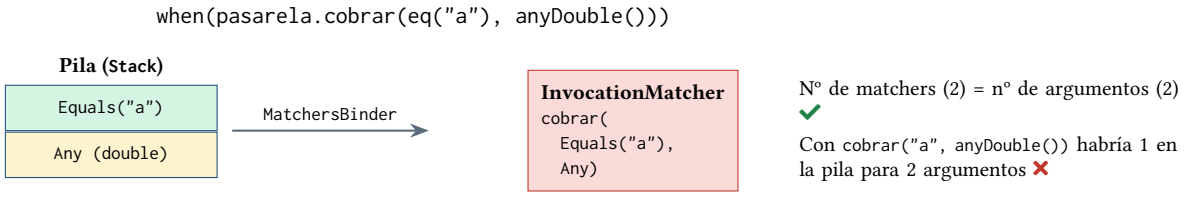


Figura 4.5: Los comodines viajan por una pila, no dentro de los argumentos

`ArgumentCaptor` es un caso especial: `captor.capture()` apila un `CapturingMatcher` que acepta cualquier valor *y además lo guarda*. Así, después de `verify`, `captor.getValue()` devuelve el argumento real.

4.6.10 La libreta: `InvocationContainer`

Cada mock tiene un `InvocationContainerImpl` con dos listas:

- **Invocaciones registradas** (`DefaultRegisteredInvocations`): todo lo que se le pidió al mock, en orden. Es lo que revisa `verify`.
- **Stubs** (`LinkedList<StubbedInvocationMatcher>`): pares *InvocationMatcher* → *cola de Answers*. Cada stub nuevo se inserta **al principio** (`addFirst`), y la búsqueda (`findAnswerFor`) recorre la lista desde el principio. Por eso **el último stub gana**.

Además, al convertir una invocación en stub, `addAnswer` **borra del registro** la llamada "de ensayo" hecha dentro de `when(...)` (`removeLast`), para que no cuente en las verificaciones. `E09` comprueba ambos comportamientos.

Cuando un stub tiene varias respuestas (`thenReturn(3, 2, 1)`), la cola de `Answer` se consume una por una y la última se repite.

4.6.11 Las respuestas: `Answer`

Todo lo que un mock devuelve lo decide un objeto `Answer`. `thenReturn(5)` crea internamente un `Returns(5)`; `thenThrow(e)` crea un `ThrowsException(e)`; `doNothing()`, un `DoesNothing`; `thenCallRealMethod()`, un `CallsRealMethods`. Si no hay stub, se usa la **respuesta por defecto** del mock:

Tabla 4.6: Respuestas por defecto (`org.mockito.Answers`)

Respuesta	Comportamiento
`RETURNS_DEFAULTS`	(predeterminada, clase `ReturnsEmptyValues`) `0`/`false` para primitivos y sus envoltorios, colecciones vacías, `Optional.empty()`, `Stream.empty()`; `null` para el resto.
`RETURNS_SMART_NULLS`	En lugar de `null`, devuelve un objeto que, si se usa, lanza una excepción que indica *qué llamada* lo produjo.
`RETURNS_MOCKS`	Devuelve valores vacíos o, si no es posible, otro mock.
`RETURNS_DEEP_STUBS`	Devuelve mocks encadenados: permite `when(a.getB().getC()).thenReturn(x)`.
`CALLS_REAL_METHODS`	Ejecuta el método real (es lo que usan los *spies*).
`RETURNS_SELF`	Devuelve el propio mock cuando el tipo coincide (útil para *builders*).

`Answer` es también un **punto de extensión**: cualquier clase que implemente `answer(InvocationOnMock)` puede usarse como respuesta. Ejemplo: `StockSegunCodigo` en `E10`.

4.6.12 La verificación: `VerificationMode`

`verify(mock, times(2))` no verifica nada todavía: `MockitoCore.verify` envuelve el modo en un `MockAwareVerificationMode`, lo deja en `MockingProgress` ("la próxima llamada a *este* mock es una verificación") y devuelve el mismo mock. Cuando el test llama a `.descontarStock("P1", 2)` sobre ese mock, el manejador ve el modo pendiente y ejecuta la verificación.

Tabla 4.7: Modos de verificación y sus clases

API	Clase	Qué comprueba
`times(n)`, `never()`	`Times`	Exactamente n invocaciones que coinciden.
`atLeast(n)` / `atMost(n)`	`AtLeast` / `AtMost`	Al menos / como mucho n.
`only()`	`Only`	Es la única invocación del mock.
`verifyNoInteractions`	`NoInteractions`	El mock no recibió ninguna llamada.
`verifyNoMoreInteractions`	`NoMoreInteractions`	Todas las llamadas ya fueron verificadas.
`inOrder.verify`	`InOrderWrapper`	Orden relativo, usando el número de secuencia.
`timeout(ms)`, `after(ms)`	`Timeout`, `After`	Para código asíncrono: espera hasta que se cumpla.

Para contar, `InvocationsFinder` busca en el `InvocationContainer` las invocaciones que coinciden con el `InvocationMatcher`. Las que se verifican quedan **marcadas** (así sabe `verifyNoMoreInteractions` cuáles faltan). Si el conteo falla, se genera el error adecuado: `WantedButNotInvoked`, `TooFewActualInvocations`, `TooManyActualInvocations` o `NeverWantedButInvoked`. Si hay una llamada *parecida* con otros argumentos, se genera `ArgumentsAreDifferent`, que muestra la diferencia.

4.6.13 Los mensajes: `Reporter` y limpieza de trazas

Una de las razones de la popularidad de Mockito son sus mensajes de error, que explican qué pasó y cómo corregirlo. Todos se construyen en `org.mockito.internal.exceptions.Reporter`. Además, `DefaultStackTraceCleaner` (plugin `StackTraceCleanerProvider`) elimina de la traza de la excepción las líneas internas de Mockito y Byte Buddy, para que el primer renglón apunte a *tu* código.

4.7 Recorrido paso a paso: qué pasa en cada línea

Se sigue ahora una prueba completa y se muestra qué componente actúa en cada momento.

```
1 InventarioRepository inv = mock(InventarioRepository.class);   // (A) creación
2 when(inv.stockDisponible("P1")).thenReturn(5);                 // (B) stubbing
3 int stock = inv.stockDisponible("P1");                         // (C) uso (lo haría el SUT)
4 inv.descontarStock("P1", 2);
5 verify(inv).descontarStock("P1", 2);                           // (D) verificación
```

Código 4.1: Prueba de referencia para el recorrido

4.7.1 (A) Creación del mock

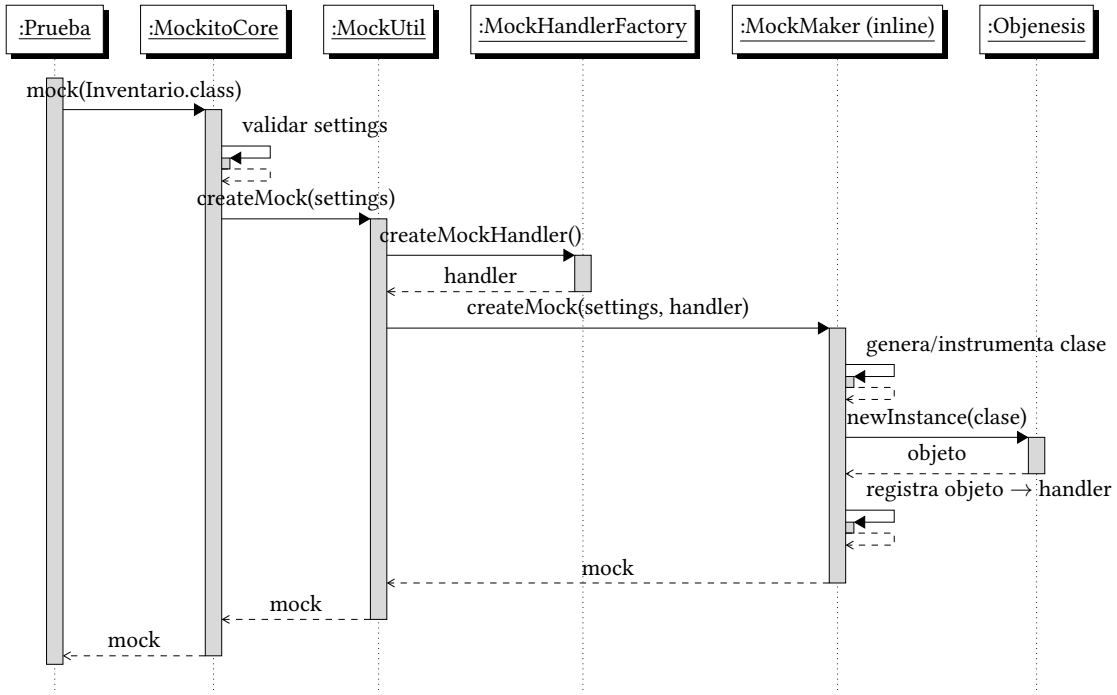


Figura 4.6: Secuencia de creación de un mock

1. `Mockito.mock` delega en `MockitoCore.mock`, que arma y **valida** la configuración (por ejemplo, rechaza tipos marcados con `@DoNotMock` o combinaciones inválidas).
2. `MockUtil.createMock` crea la cadena de `MockHandler`.
3. El `MockMaker` inline genera una clase que implementa la interfaz (o reescribe la clase si es concreta). Byte Buddy guarda las clases generadas en caché, así que el segundo mock del mismo tipo es mucho más rápido.
4. Objenesis crea el objeto sin constructor.
5. El `MockMaker` anota en su mapa que ese objeto es un mock y cuál es su manejador.

4.7.2 (B) Stubbing: `when(inv.stockDisponible("P1")).thenReturn(5)`

Java evalúa las expresiones **de dentro hacia fuera**. Primero se ejecuta `inv.stockDisponible("P1")` y solo después `when(...)`.

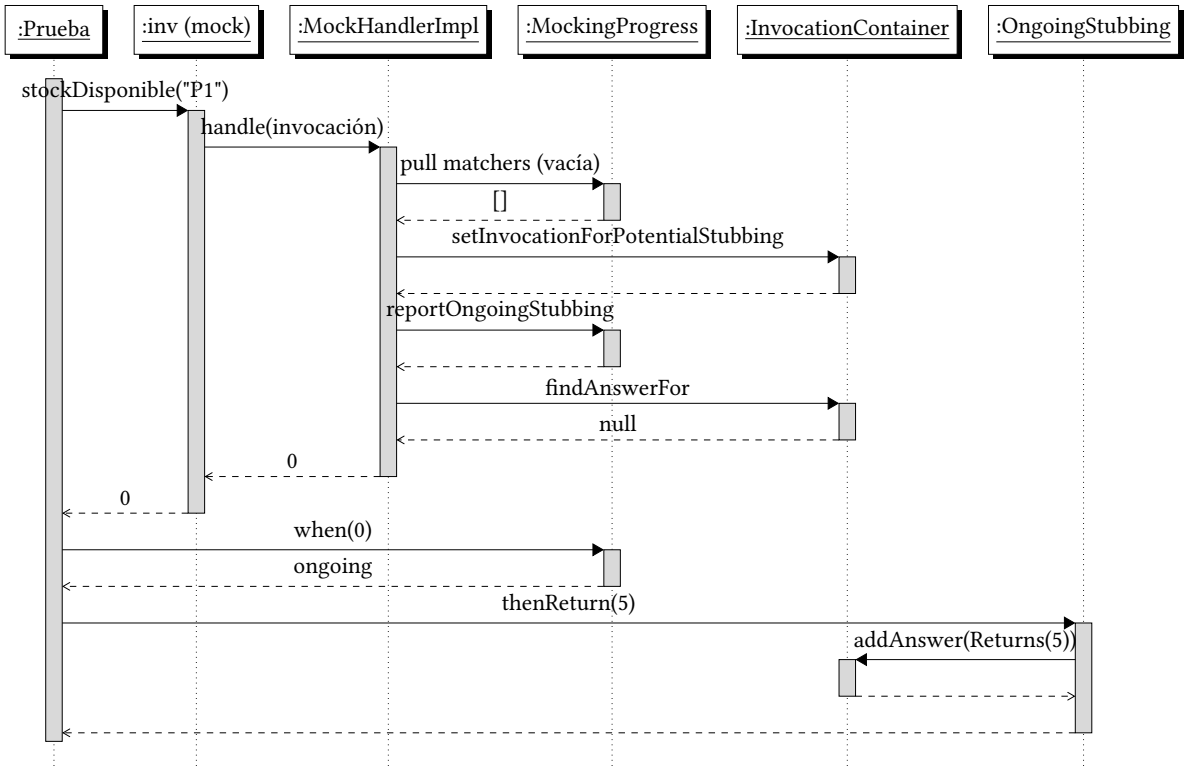


Figura 4.7: Secuencia de `when(...).thenReturn(...)`

1. **La llamada de ensayo.** `inv.stockDisponible("P1")` llega al `MockHandlerImpl`.
2. **Comodines.** `MatchersBinder` vacía la pila de matchers. Está vacía, así que `"P1"` se envuelve en `Equals("P1")` y se arma el `InvocationMatcher` `stockDisponible(Equals("P1"))`.
3. **Validación de estado.** `MockingProgress.validateState()` comprueba que no quedó nada a medias (por ejemplo, un `when` sin `thenReturn`).
4. **¿Es una verificación?** No hay modo de verificación pendiente, así que es una llamada normal.
5. **Registro.** La invocación se guarda en el `InvocationContainer` como "posible stubbing".
6. **Aviso al apuntador.** Se crea un `OngoingStubbingImpl` y se deja en `MockingProgress` como *última invocación*.
7. **Respuesta.** Se busca un stub; no hay ninguno, así que responde el `RETURNS_DEFAULTS`: `0`.
8. **`when(0)`.** El argumento `0` se ignora. `MockitoCore.when` marca "stubbing en curso" y **toma** el `OngoingStubbing` pendiente de `MockingProgress`. Si no hubiera ninguno, lanzaría `MissingMethodInvocationException`.
9. **`thenReturn(5)`.** Crea un `Returns(5)` y llama a `InvocationContainer.addAnswer`, que (a) borra la llamada de ensayo del registro, (b) valida que `5` sea compatible con el tipo de retorno (si no, `WrongTypeOfReturnValue`) y (c) inserta el stub al inicio de la lista.

Por qué los métodos `void` usan `doXxx()`

`when()` necesita recibir *un valor* (el resultado de la llamada de ensayo). Un método `void` no devuelve nada, así que `when(notificador.enviar(...))` no compila. La familia `doThrow()/doAnswer()/doNothing()/doReturn()` invierte el orden: `doThrow(e).when(notificador)` deja las respuestas preparadas en el `InvocationContainer` del mock (`setAnswersForStubbing`) y devuelve el mock. La siguiente llamada, `.enviar(...)`, es detectada por el manejador (`hasAnswersForStubbing()`) y convertida directamente en stub, sin ejecutar nada.

4.7.3 (C) Uso del mock desde el código bajo prueba

Cuando el SUT llama a `inv.stockDisponible("P1")`, se repiten los pasos 1 a 6. En el paso 7, `findAnswerFor` recorre la lista de stubs, encuentra que `stockDisponible(Equals("P1"))` coincide y ejecuta su `Answer`: devuelve 5. Antes, si el stub tiene un `ArgumentCaptor`, captura los argumentos. Con `STRICT_STUBS`, el *listener* de estrictez anota que ese stub **fue usado**. Si encuentra un stub del mismo método pero con *otros* argumentos, lanza `PotentialStubbingProblem`.

La llamada a `descontarStock("P1", 2)` no tiene stub y es `void`: se registra y no hace nada.

4.7.4 (D) Verificación

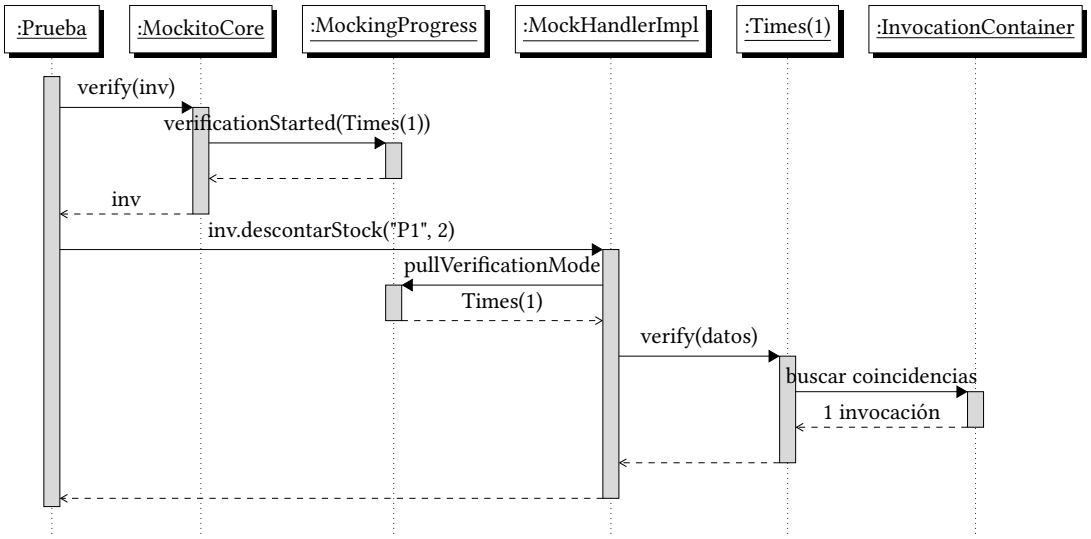


Figura 4.8: Secuencia de `verify(mock).metodo(...)`

1. `verify(inv)` comprueba que `inv` sea un mock (si no, `NotAMockException`), deja `Times(1)` pendiente en `MockingProgress` y devuelve el propio `inv`.
2. La llamada `.descontarStock("P1", 2)` llega al manejador, que toma el modo pendiente y comprueba que se refiere a *este* mock.
3. Se construye el `InvocationMatcher` (con los comodines, si los hay) y `Times` cuenta las invocaciones registradas que coinciden. Hay 1, así que la verificación pasa y la invocación queda marcada como verificada. Esta llamada **no** se registra como un uso más del mock.

4.8 Ciclo de vida de una prueba con `MockitoExtension`

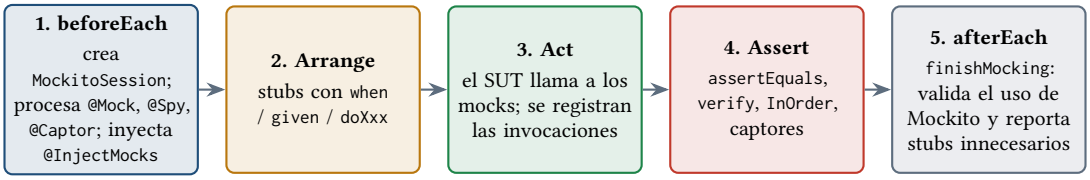


Figura 4.9: Ciclo de vida de una prueba con `MockitoExtension`

`MockitoExtension` (del módulo `mockito-junit-jupiter`) se engancha a JUnit mediante sus *callbacks*:

- **`beforeEach`**: ejecuta

  ```
  Mockito.mockitoSession().initMocks(this).strictness(STRICT_STUBS).startMocking()
  ```

  La estrictez se puede cambiar con `@MockitoSettings(strictness = ...)`.
- **Resolución de parámetros**: permite recibir mocks como parámetros del método de prueba (`void prueba(@Mock Notificador n)`).
- **`afterEach`**: llama a `session.finishMocking(errorDeLaPrueba)`, que valida que no quedó un stubbing a medias ni matchers sueltos y, si la prueba pasó, reporta los stubs no usados.

4.8.1 Procesamiento de anotaciones e inyección

`initMocks` usa el componente `InjectingAnnotationEngine`, que trabaja en dos fases:

1. **Creación** (`IndependentAnnotationEngine + SpyAnnotationEngine`): para cada campo con `@Mock` llama a `MockAnnotationProcessor`, que traduce los atributos de la anotación (`name`, `answer`, `strictness`, `mockMaker`…) a un `MockSettings` y crea el mock. `@Captor` crea un `ArgumentCaptor` y `@Spy` envuelve la instancia del campo, o la crea con el constructor sin argumentos si el campo está vacío.
2. **Inyección** (`DefaultInjectionEngine`) sobre cada campo `@InjectMocks`, probando estrategias en orden:
   a) **`ConstructorInjection`**: elige el constructor con **más parámetros** y pasa los mocks cuyo tipo coincide (`null` si no hay mock para un parámetro). Si tiene éxito, *no se aplican* las demás estrategias. Es la que se usa con `PedidoService`.
   b) **`PropertyAndSetterInjection`**: si la clase no tiene ese constructor, la crea con el constructor sin argumentos y asigna los mocks por *setter* o directamente al campo, primero por **tipo** y, si hay ambigüedad, por **nombre**.

⚠ La inyección falla en silencio

Si `@InjectMocks` no encuentra cómo inyectar un mock, no lanza un error: el campo queda en `null` y la prueba falla después con un `NullPointerException`. Por eso se recomienda la inyección por constructor, que es explícita.

4.9 Strictez: el "control de calidad" de las pruebas

El componente `Strictness` define cuán exigente es Mockito con los stubs:

Tabla 4.8: Niveles de estrictez

Nivel	Comportamiento
`LENIENT`	Permisivo: no se queja de nada (comportamiento clásico de Mockito 1).
`WARN`	Imprime avisos de stubs no usados o con argumentos distintos.
`STRICT_STUBS`	(por defecto con `MockitoExtension`) Hace fallar la prueba si (a) un stub nunca se usó: `UnnecessaryStubbingException`, al final; o (b) el código llamó al método con argumentos distintos de los programados: `PotentialStubbingProblem`, en el momento. También hace que `verifyNoMoreInteractions` ignore las llamadas ya "cubiertas" por stubs.

Técnicamente, un *listener* (`DefaultStubbingLookupListener`) se entera de cada búsqueda de stub en el paso 7 del recorrido y marca los stubs usados. Al terminar la prueba, `UniversalTestListener` revisa cuáles quedaron sin usar. `lenient()` marca un stub individual como exento.

4.10 Mocks estáticos y de constructores

Estas dos capacidades solo existen con el *mock maker* inline:

- **`mockStatic(Clase.class)`**: el *advice* insertado en los métodos estáticos consulta un registro **por hilo** de "clases con estáticos simulados". Devuelve un `MockedStatic` que, al cerrarse (`close()` o fin del `try`), elimina la clase del registro. Fuera del bloque, o en otro hilo, el método vuelve a ser real.
- **`mockConstruction(Clase.class)`**: el *advice* en los constructores hace que cada `new Clase()` dentro del bloque produzca un mock. Se puede configurar cada mock creado con una función de inicialización (`E10`).

⚠ Por qué deben cerrarse

Si un `MockedStatic` no se cierra, la simulación sigue activa en ese hilo y contamina las pruebas siguientes. Mockito lanza un error si se intenta registrar dos veces la misma clase en el mismo hilo.

4.11 Extensibilidad: el sistema de plugins

Mockito permite reemplazar componentes sin modificar su código. Hay tres niveles:

1. **Por mock**: `withSettings().mockMaker(MockMakers.SUBCLASS)`, `.defaultAnswer(...)`, `.invocationListeners(...)`, o los atributos equivalentes de `@Mock`.
2. **Por componente de usuario**: implementar `Answer` o `ArgumentMatcher` (`E10`).
3. **Global, por plugin**: crear en `src/test/resources/mockito-extensions/` un archivo con el nombre de la interfaz del plugin y, dentro, la clase o el alias que la implementa. `PluginLoader` y `PluginRegistry` leen esos archivos al iniciar. Si no encuentran ninguno, usan las implementaciones por defecto (`DefaultMockitoPlugins`).

```
# archivo: src/test/resources/mockito-extensions/org.mockito.plugins.MockMaker
mock-maker-subclass          # alias: mock-maker-inline | mock-maker-subclass | mock-maker-proxy
```

Código 4.2: Cambiar globalmente el `MockMaker` mediante el sistema de plugins

Tabla 4.9: Plugins disponibles (`org.mockito.plugins`)

Plugin	Qué se puede reemplazar
`MockMaker`	La forma de fabricar mocks.
`InstantiatorProvider2`	Cómo se instancian los objetos (Objenesis por defecto).
`AnnotationEngine`	El procesamiento de `@Mock`, `@Spy`, etc.
`StackTraceCleanerProvider`	Qué líneas se eliminan de las trazas de error.
`MemberAccessor`	Cómo se leen y escriben campos por reflexión.
`DoNotMockEnforcer`	Qué tipos está prohibido simular (anotación `@DoNotMock`).
`MockitoLogger`	Dónde se escriben los avisos.
`MockResolver`	Cómo reconocer un mock envuelto por otro framework (p. ej., proxies de Spring).
`PluginSwitch`	Activar o desactivar plugins según condiciones.

4.12 Demostración práctica de la arquitectura

Los ejemplos `E09` y `E10` (14 pruebas) hacen visibles los componentes. Esta es la salida real al ejecutarlos:

```
--- Radiografía del mock ---
¿Es mock?        true
Tipo simulado:   InventarioRepository
Respuesta def.:  RETURNS_DEFAULTS
MockHandler:     InvocationNotifierHandler
Invocaciones:    [inventarioRepository.stockDisponible("P1");, inventarioRepository.
    descontarStock("P1", 2);]
Stubs:           [inventarioRepository.stockDisponible("P1"); stubbed with: [Returns: 5]]
Clase del mock de clase concreta: bo.edu.umss.pedidos.servicio.CalculadoraDescuentos
Clase del mock de interfaz:       bo.edu.umss.pedidos.puertos.
    InventarioRepository$MockitoMock$YWsjgKQa
INLINE   -> bo.edu.umss.pedidos.servicio.CalculadoraDescuentos
SUBCLASS -> bo.edu.umss.pedidos.servicio.CalculadoraDescuentos$MockitoMock$oe8a1izx
PROXY    -> jdk.proxy2.$Proxy87
notificador.enviar("a@umss.edu.bo", "uno");
notificador.enviar("b@umss.edu.bo", "dos");
```

Código 4.3: Salida de `E09` y `E10` (consola)

Tabla 4.10: Qué componente demuestra cada prueba

Prueba	Componente y comportamiento demostrado
E09_ArquitecturaInternaTest	
Radiografía de un mock	`MockingDetails`: settings, cadena de `MockHandler`, invocaciones y stubs del `InvocationContainer`.
Llamada de ensayo	`InvocationContainer.addAnswer` borra la invocación hecha dentro de `when()`.
El último stub gana	La lista de stubs se llena con `addFirst`.
Inline no crea subclase	`MockMaker` inline: el mock de una clase concreta tiene la misma clase; su código real no se ejecuta.
Spy copia el estado	`LenientCopyTool` copia los campos; el objeto original no cambia.
`when()` toma la última invocación	`MockingProgress` (`ThreadLocal`) guarda la última invocación de *cualquier* mock.
`when()` sin invocación	`MockitoCore.when` lanza `MissingMethodInvocationException`.
Pila de matchers	`ArgumentMatcherStorage` + `MatchersBinder`: "2 matchers expected, 1 recorded".
E10_ComponentesExtensiblesTest	
Answer propio	Punto de extensión `Answer` como respuesta por defecto.
ArgumentMatcher propio	Punto de extensión `ArgumentMatcher` con `toString()` descriptivo.
Tres MockMaker	`MockMakers.INLINE`, `SUBCLASS`, `PROXY`: clase resultante de cada uno.
Subclass y `final`	El *mock maker* subclass no puede simular clases `final`; el inline sí.
InvocationListener	`InvocationNotifierHandler` avisa a los listeners de cada llamada.
`mockConstruction`	`new` dentro del bloque devuelve un mock; fuera vuelve a ser real.

4.13 Diagnóstico: qué componente produce cada error

Conocer la arquitectura permite entender los errores de Mockito en lugar de memorizarlos:

Tabla 4.11: Errores frecuentes, componente que los detecta y causa

Excepción	Componente	Causa y solución
`MissingMethodInvocation Exception`	`MockitoCore.when`	`when()` no encontró ninguna invocación pendiente: el objeto no es un mock, o el método es `final`/`static` con un *mock maker* que no lo soporta.
`UnfinishedStubbing Exception`	`MockingProgress`	Un `when(...)` sin `thenReturn`, o se llamó a otro mock dentro de `thenReturn(...)`.
`InvalidUseOfMatchers Exception`	`MatchersBinder`	Se mezclaron valores y comodines, o se usó un comodín fuera de `when`/`verify`. Todos los argumentos deben ser comodines (usar `eq()`).
`NullPointerException` con `any()`	Java + pila de matchers	`any()`/`argThat()` devuelven `null`, que no se puede convertir a primitivo. Usar `anyInt()`, `doubleThat()`…
`WrongTypeOfReturnValue`	`InvocationContainer`	El valor de `thenReturn` no es compatible con el tipo de retorno, o `when()` tomó una invocación pendiente de otro método (ver `E09`).
`UnnecessaryStubbing Exception`	Estrictez: `UniversalTestListener`	Un stub nunca se usó. Borrarlo o marcarlo con `lenient()`.

Excepción	Componente	Causa y solución
`PotentialStubbingProblem`	Estrictez: `DefaultStubbingLookupListener`	El SUT llamó al método con argumentos distintos de los programados.
`WantedButNotInvoked`	`Times` / `InvocationsFinder`	Se esperaba una llamada que nunca ocurrió.
`ArgumentsAreDifferent`	`Times`	Hubo una llamada al método, pero con otros argumentos; el mensaje muestra la diferencia.
`TooManyActualInvocations` / `TooFewActualInvocations`	`Times`	El número de llamadas no coincide con `times(n)`.
`NoInteractionsWanted`	`NoMoreInteractions`	Quedan llamadas sin verificar.
`NotAMockException`	`MockitoCore.verify`	Se pasó a `verify` un objeto real.
`MockitoException: Cannot mock/spy class`	`MockMaker`	Tipo no simulable con el *mock maker* actual (p. ej., clase `final` con *subclass*), o tipo marcado con `@DoNotMock`.

4.14 Resumen del capítulo

Mockito en cinco ideas

1. Un **mock** es un objeto fabricado en ejecución (por Byte Buddy y Objenesis, a través de un `MockMaker`) cuyos métodos no ejecutan código real: **todas las llamadas van a su `MockHandler`**.
2. El `MockHandler` **registra** cada llamada en el `InvocationContainer` y **responde** con el `Answer` del stub que coincide o, si no hay, con la respuesta por defecto.
3. `when()`, `verify()` y los comodines se coordinan a través de `MockingProgress`, una memoria **por hilo** que recuerda la última invocación, el modo de verificación pendiente y la pila de matchers.
4. Las reglas "raras" de Mockito (todos los argumentos con matchers, `doXxx` para `void`, `doubleThat` para primitivos) son **consecuencias directas** de este diseño.
5. La arquitectura es **extensible**: el *mock maker*, las respuestas, los comodines y otros componentes se pueden reemplazar mediante `MockSettings` o el sistema de plugins.

Capítulo 5

SRS – Especificación de Requisitos del Sistema de Ejemplo

Antes de diseñar (SDD) y antes de probar hay que saber **qué debe hacer el sistema**. Ese es el papel de la **SRS** (*Software Requirements Specification*). Este capítulo sigue la estructura de la norma **IEEE 830-1998**, actualizada por **ISO/IEC/IEEE 29148:2018** (*Requirements engineering*).

¿Por qué la SRS es clave para las pruebas?

La SRS es la **base de prueba** de la caja negra: cada caso de prueba de caja negra nace de una frase de este documento. Por eso cada requisito se redacta de forma **verificable**, con un identificador único (RF-xx) y un **criterio de aceptación**. Así, cualquier persona puede comprobar si el sistema lo cumple.

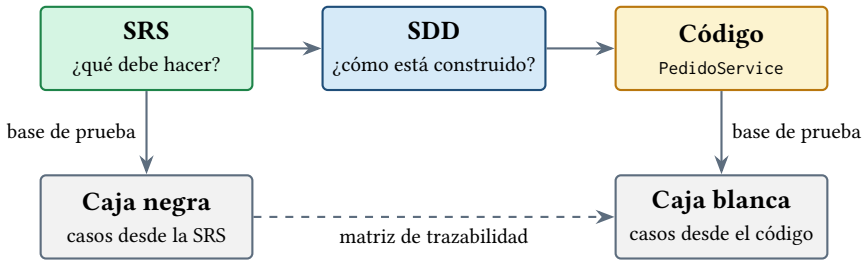


Figura 5.1: Relación entre SRS, SDD, código y pruebas

5.1 Introducción

5.1.1 Propósito

Especificar los requisitos funcionales y no funcionales del *Servicio de Pedidos*, un componente de una tienda en línea que procesa compras. Está dirigido al equipo de desarrollo (que lo implementa), al equipo de pruebas (que diseña los casos de caja negra a partir de él) y al auditor (que verifica la trazabilidad).

5.1.2 Alcance

El producto se llama **Servicio de Pedidos** (`PedidoService`). Recibe un pedido, verifica que haya stock, calcula el descuento, cobra al cliente mediante una pasarela de pagos externa, actualiza el inventario, registra el pedido y notifica al cliente por correo. **No** incluye la interfaz de usuario, la base de datos, el banco ni el servidor de correo, que son sistemas externos con los que el servicio se comunica mediante interfaces.

5.1.3 Definiciones, acrónimos y abreviaturas

Pedido Solicitud de compra de un cliente, con uno o más *items*.

Item Línea de un pedido: un producto y la cantidad solicitada.

Subtotal Suma de *precio* × *cantidad* de todos los items, en bolivianos (Bs).

Descuento Monto en Bs que se resta del subtotal según las reglas RN-01 y RN-02.

Total Subtotal − descuento. Es el monto que se cobra.

Cliente VIP Cliente con beneficios adicionales de descuento.

Stub / Mock Dobles de prueba (ver capítulo 4).

RF / RNF / RN / CU / IE Requisito funcional / no funcional / regla de negocio / caso de uso / interfaz externa.

5.1.4 Referencias

IEEE 830-1998; ISO/IEC/IEEE 29148:2018; IEEE 1016-2009 (SDD, capítulo 6).

5.1.5 Visión general del documento

La sección 2 describe el producto de forma general. La sección 3 detalla las interfaces externas, los requisitos funcionales con sus criterios de aceptación, las reglas de negocio, los casos de uso y los requisitos no funcionales. La sección 4 registra las observaciones de la revisión.

5.2 Descripción general

5.2.1 Perspectiva del producto

El Servicio de Pedidos es un componente de la *capa de negocio*. Lo invoca una aplicación web y, a su vez, usa cuatro servicios externos: el inventario y el registro de pedidos (base de datos), la pasarela de pagos (banco) y el notificador (correo). El diagrama de contexto se presenta en el SDD (figura 6.1).

5.2.2 Funciones del producto

F1. Validar que el pedido tenga productos.

F2. Verificar el stock de cada producto.

F3. Calcular el descuento según el monto y el tipo de cliente.

F4. Cobrar el total al cliente.

F5. Actualizar el inventario, registrar el pedido y notificar al cliente.

F6. Consultar un pedido existente.

5.2.3 Actores

Tabla 5.1: Actores del sistema

Actor	Descripción
Aplicación web (cliente)	Envía el pedido y el tipo de cliente; recibe el pedido procesado o un error.
Pasarela de pagos	Sistema externo del banco; aprueba o rechaza cobros.
Sistema de inventario	Informa del stock y lo descuenta.
Servidor de correo	Entrega las notificaciones al cliente.

5.2.4 Restricciones

- Lenguaje Java 21 o superior; las pruebas usan JUnit 6 y Mockito 5.
- Los montos se expresan en bolivianos y se redondean a 2 decimales.
- Las pruebas unitarias no pueden acceder a sistemas reales (RNF-03).

5.2.5 Suposiciones y dependencias

- La pasarela de pagos responde siempre con “aprobado” o “rechazado” y un mensaje.
- El inventario informa el stock actual de un producto por su código.
- Las cantidades de cada item son enteros positivos (ver observación OBS-02).

5.3 Requisitos específicos

5.3.1 Interfaces externas

Tabla 5.2: Interfaces externas (serán los puntos de sustitución por mocks)

ID	Interfaz	Operaciones
IE-01	Inventario	`stockDisponible(codigo) : int`; `descontarStock(codigo, cantidad)`
IE-02	Pasarela de pagos	`cobrar(email, monto) : ResultadoPago(aprobado, codigoTransaccion, mensaje)`
IE-03	Notificador	`enviar(destinatario, mensaje)`
IE-04	Registro de pedidos	`guardar(pedido) : Pedido`; `buscarPorId(id) : Optional<Pedido>`

5.3.2 Requisitos funcionales

Cada requisito se presenta como una ficha con su criterio de aceptación en formato *Dado / Cuando / Entonces*, que es directamente convertible en un caso de prueba.

RF-01 – Rechazar pedidos sin productos

Descripción	El sistema debe rechazar un pedido que no contenga ningún item.
Entradas	Pedido con lista de items vacía.
Proceso	Validar la cantidad de items antes de cualquier otra operación.
Salidas	Error “El pedido no tiene items”. No se consulta el inventario ni se cobra.
Prioridad	Alta
Criterio de aceptación	**Dado** un pedido sin items, **cuando** se procesa, **entonces** se produce un error de pedido inválido y no se realiza ningún cobro.

RF-02 – Verificar stock antes de cobrar

Descripción	Antes de cobrar, el sistema debe comprobar que el stock de *cada* producto sea suficiente para la cantidad pedida (RN-03).
Entradas	Items del pedido; stock informado por IE-01.
Proceso	Para cada item, comparar stock disponible con cantidad.
Salidas	Si algún item no tiene stock suficiente: error “Stock insuficiente para el producto X”.
Prioridad	Alta
Criterio de aceptación	**Dado** un item con cantidad 2 y stock 1, **cuando** se procesa el pedido, **entonces** se produce el error de stock insuficiente y no se cobra. **Dado** stock = cantidad, **entonces** el stock se considera suficiente.

RF-03 – Calcular el descuento

Descripción	El sistema debe calcular el descuento del pedido según RN-01 y RN-02.
Entradas	Subtotal (Bs) y tipo de cliente (REGULAR o VIP).
Proceso	Determinar el porcentaje según el tramo del subtotal; sumar 5 % si es VIP; multiplicar y redondear.
Salidas	Monto del descuento en Bs con 2 decimales. Si el subtotal es negativo: error.
Prioridad	Alta
Criterio de aceptación	**Dado** un subtotal de 500 Bs y un cliente REGULAR, **cuando** se calcula el descuento, **entonces** es 50,00 Bs. **Dado** el mismo subtotal y un cliente VIP, **entonces** es 75,00 Bs.

RF-04 – Pago rechazado

Descripción	Si la pasarela rechaza el cobro, el pedido debe quedar en estado RECHAZADO, se debe notificar al cliente con el motivo y **no** se debe modificar el inventario ni registrar el pedido.
Entradas	Respuesta de IE-02 con `aprobado = false` y un mensaje.
Proceso	Cambiar el estado y enviar la notificación.
Salidas	Pedido en estado RECHAZADO; correo "Pago rechazado: <motivo>".
Prioridad	Alta
Criterio de aceptación	**Dado** un pedido válido con stock y un banco que rechaza con "Sin fondos", **cuando** se procesa, **entonces** el estado es RECHAZADO, se envía "Pago rechazado: Sin fondos" y no se descuenta stock.

RF-05 – Pago aprobado

Descripción	Si la pasarela aprueba el cobro del total, el sistema debe descontar el stock de cada item, marcar el pedido como PAGADO con su total, registrarlo y enviar una confirmación con el código de transacción.
Entradas	Respuesta de IE-02 con `aprobado = true` y código de transacción.
Proceso	Descontar stock (IE-01), registrar (IE-04), notificar (IE-03).
Salidas	Pedido PAGADO con total = subtotal − descuento; correo de confirmación.
Prioridad	Alta
Criterio de aceptación	**Dado** un pedido de 2 monitores de 250 Bs para un cliente REGULAR y un banco que aprueba, **cuando** se procesa, **entonces** se cobran 450 Bs, el estado es PAGADO, se descuentan 2 unidades y se envía la confirmación.

RF-06 – Consultar un pedido

Descripción	El sistema debe devolver un pedido a partir de su identificador.
Entradas	Identificador del pedido.
Proceso	Buscar en IE-04.
Salidas	El pedido, o el error "No existe el pedido <id>".
Prioridad	Media
Criterio de aceptación	**Dado** un identificador inexistente, **cuando** se consulta, **entonces** se produce el error de pedido no encontrado.

5.3.3 Reglas de negocio

Tabla 5.3: Reglas de negocio

ID	Regla
RN-01	Porcentaje de descuento por tramo de subtotal: menor que 100 Bs → 0 %; de 100 a 499,99 Bs → 5 %; 500 Bs o más → 10 %. Un subtotal negativo es inválido.
RN-02	Un cliente VIP recibe 5 puntos porcentuales adicionales en cualquier tramo.
RN-03	El stock es suficiente si *stock disponible* ≥ *cantidad pedida*.
RN-04	Total = subtotal − descuento. Todos los montos se redondean a 2 decimales.

5.3.4 Casos de uso

CU-01 – Procesar pedido

Actor principal: aplicación web. **Actores secundarios:** inventario, pasarela de pagos, notificador.
Precondición: el pedido existe en memoria con estado CREADO.
Postcondición de éxito: pedido PAGADO, stock actualizado, pedido registrado, cliente notificado.
Flujo básico

1. La aplicación web envía el pedido y el tipo de cliente.
2. El sistema verifica que el pedido tenga items (RF-01).
3. Para cada item, el sistema consulta el stock y comprueba que sea suficiente (RF-02).
4. El sistema calcula el subtotal, el descuento (RF-03) y el total.
5. El sistema solicita a la pasarela el cobro del total.
6. La pasarela aprueba el cobro.
7. El sistema descuenta el stock de cada item, marca el pedido como PAGADO, lo registra y envía la confirmación (RF-05).

Flujos alternativos

FA-1 (paso 2): el pedido no tiene items → error; fin del caso de uso.

FA-2 (paso 3): un producto no tiene stock suficiente → error de stock; fin, sin cobrar.

FA-3 (paso 6): la pasarela rechaza el cobro → estado RECHAZADO y notificación del motivo (RF-04); fin.

CU-02 – Consultar pedido

Flujo básico: (1) se envía el identificador; (2) el sistema lo busca en el registro; (3) devuelve el pedido.
FA-1 (paso 2): no existe → error “No existe el pedido <id>” (RF-06).

5.3.5 Requisitos no funcionales

Tabla 5.4: Requisitos no funcionales

ID	Atributo	Requisito y forma de verificarlo
RNF-01	Testabilidad	Toda dependencia externa se accede mediante una interfaz recibida por constructor. *Verificación:* revisión del diseño (SDD).
RNF-02	Calidad	La capa de servicio debe tener una cobertura de ramas de al menos 90 %. *Verificación:* reporte JaCoCo.
RNF-03	Aislamiento	Las pruebas unitarias no deben acceder a red, base de datos ni correo reales. *Verificación:* todas las IE se sustituyen por mocks.
RNF-04	Rendimiento de pruebas	La suite completa de pruebas unitarias debe ejecutarse en menos de 5 segundos. *Verificación:* tiempo informado por JUnit.
RNF-05	Mantenibilidad	Cada requisito funcional debe tener al menos una prueba automatizada trazable. *Verificación:* matriz de trazabilidad (SDD).

5.4 Observaciones de la revisión de la SRS

Al diseñar las pruebas se hizo una **revisión de calidad** de la SRS: cada requisito debe ser completo, no ambiguo y verificable. Esa revisión encontró estos vacíos, que se registran como hallazgos:

Tabla 5.5: Hallazgos de la revisión de la SRS

ID	Hallazgo	Recomendación
OBS-01	La SRS no dice qué hacer si el pedido es *nulo*. El código lo rechaza (programación defensiva), pero ese comportamiento no está especificado. Se detectó en la caja blanca (camino C1).	Añadir a RF-01: "pedidos nulos o sin productos".
OBS-02	No se especifica qué pasa con cantidades 0 o negativas. El código no las valida: una cantidad negativa produce un subtotal negativo y el error aparece en el cálculo de descuento.	Añadir un requisito de validación de cantidades (≥ 1).
OBS-03	No se especifica el comportamiento si el tipo de cliente es nulo. El código lo trata como REGULAR.	Definir si es un error o un valor por defecto.
OBS-04	No se define qué hacer si el registro del pedido falla *después* de cobrar.	Definir una compensación (reembolso) o una transacción.

Relevancia para la auditoría

Estos hallazgos muestran que diseñar pruebas también sirve para **auditar los requisitos**: al intentar escribir un caso de prueba, el tester descubre situaciones que nadie especificó.

Capítulo 6

SDD – Documento de Diseño del Sistema de Ejemplo

Este capítulo sigue la estructura de la norma **IEEE 1016-2009** (*Software Design Descriptions*), que organiza el diseño en *puntos de vista* (*viewpoints*) dirigidos a las inquietudes de cada parte interesada. Parte de los requisitos de la SRS (capítulo 5) y describe *cómo* se construye el sistema para cumplirlos. El sistema se diseñó con la **testabilidad** (RNF-01) como requisito principal, porque su finalidad es demostrar Mockito.

6.1 Introducción

6.1.1 Propósito

Describir la arquitectura y el diseño detallado del *Servicio de Pedidos*, un componente que procesa las compras de una tienda en línea, y documentar las decisiones de diseño que permiten probarlo unitariamente con Mockito.

6.1.2 Alcance

El componente valida pedidos, verifica el stock, calcula descuentos, cobra mediante una pasarela de pagos, actualiza el inventario, guarda el pedido y notifica al cliente. La persistencia, el banco y el correo quedan fuera del alcance y se modelan como interfaces (*puertos*).

6.1.3 Definiciones

SUT *System Under Test*: clase bajo prueba (`PedidoService`).

Puerto Interfaz que abstrae un sistema externo (arquitectura hexagonal).

Mock Doble de prueba creado con Mockito.

6.1.4 Partes interesadas e inquietudes

Tabla 6.1: Partes interesadas del diseño

Parte interesada	Inquietud principal
Desarrollador	Estructura de clases, dependencias y responsabilidades.
Tester / QA	Puntos de sustitución (dónde usar mocks), caminos del código y requisitos verificables.
Auditor de sistemas	Trazabilidad requisito → prueba, evidencia de cobertura y aislamiento respecto de producción.

6.2 Requisitos de referencia

Los requisitos que este diseño debe satisfacer están en la SRS: RF-01 a RF-06, RN-01 a RN-04 y RNF-01 a RNF-05. La tabla 6.2 muestra qué elemento del diseño atiende a cada uno.

Tabla 6.2: Requisito → elemento de diseño

Requisito	Elemento de diseño que lo atiende
RF-01, RF-02, RF-04, RF-05	`PedidoService.procesarPedido` (punto de vista de algoritmo)
RF-03, RN-01, RN-02, RN-04	`CalculadoraDescuentos.calcularDescuento`
RF-06	`PedidoService.obtenerPedido` + `PedidoRepository.buscarPorId`
RNF-01	Puertos `InventarioRepository`, `PasarelaPago`, `Notificador`, `PedidoRepository` inyectados por constructor
RNF-03	Los mismos puertos: son los puntos de sustitución por mocks

6.3 Punto de vista de contexto

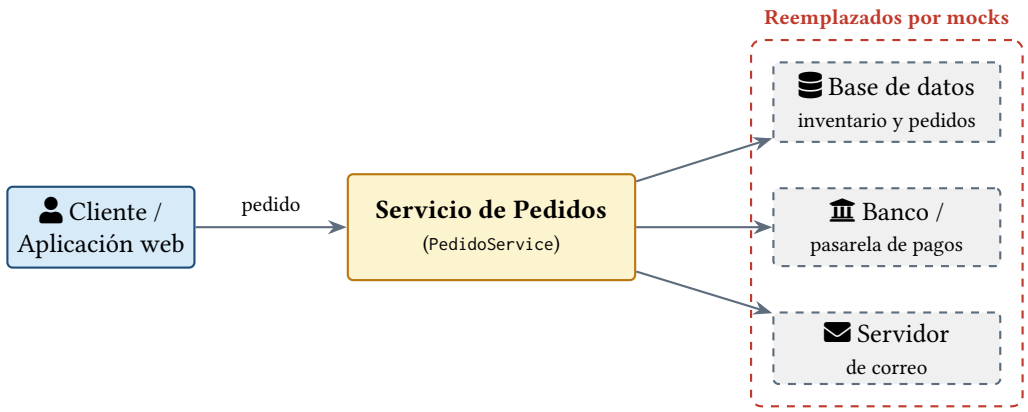


Figura 6.1: Diagrama de contexto. Los sistemas externos (línea discontinua) se sustituyen por mocks en las pruebas.

6.4 Punto de vista de composición y lógico

El diseño aplica una arquitectura en capas con *puertos* (interfaces) al estilo hexagonal. La figura 6.2 muestra el diagrama de clases.

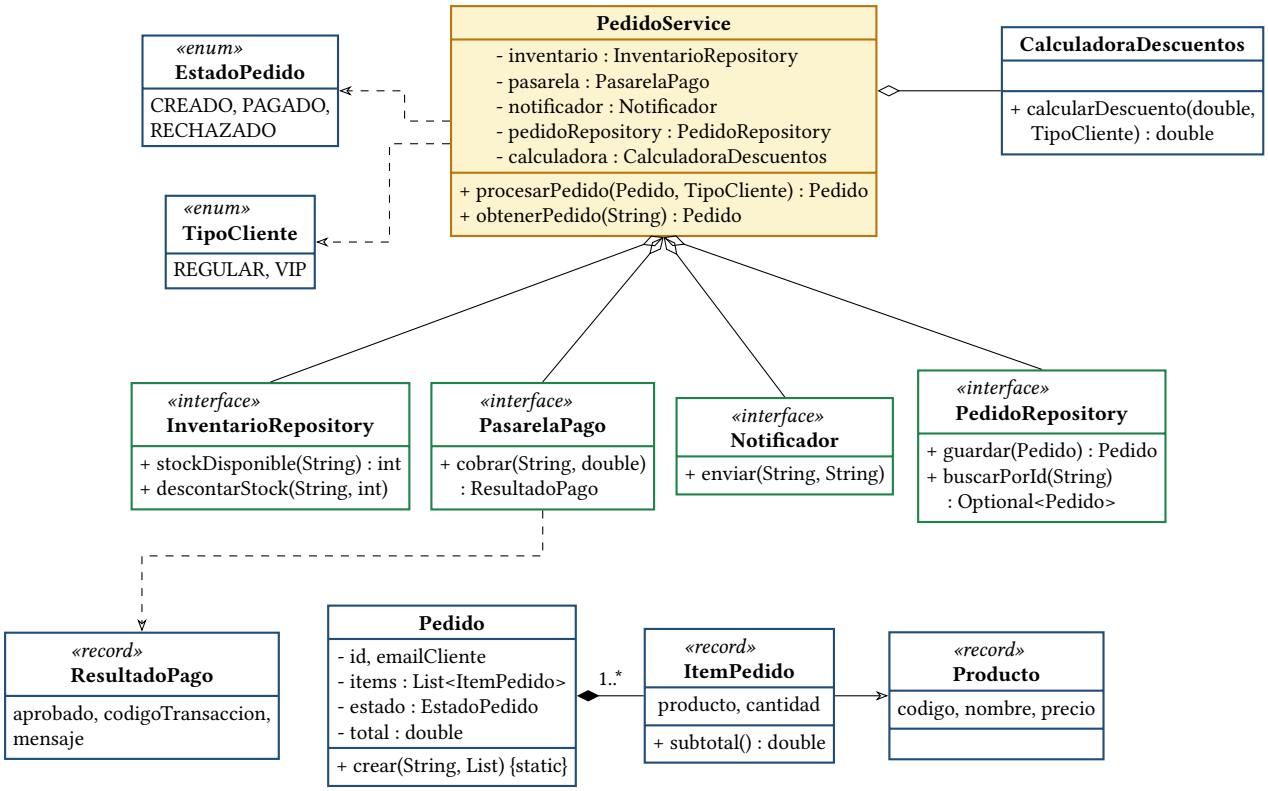


Figura 6.2: Diagrama de clases del Servicio de Pedidos (interfaces en verde = puertos *mockeables*)

Tabla 6.3: Paquetes del sistema

Paquete	Responsabilidad
`modelo`	Entidades y objetos de valor: `Pedido`, `ItemPedido`, `Producto`, `ResultadoPago`, enumeraciones y excepciones de negocio.
`puertos`	Interfaces hacia sistemas externos. **Son los puntos donde Mockito inyecta los dobles.**
`servicio`	Lógica de negocio: `PedidoService` (orquestación) y `CalculadoraDescuentos` (reglas puras).

6.5 Punto de vista de dependencias

`PedidoService` depende solo de **abstracciones**, según el principio de inversión de dependencias (la "D" de SOLID), y las recibe todas por constructor:

```
public PedidoService(InventarioRepository inventario, PasarelaPago pasarela,
                     Notificador notificador, PedidoRepository pedidoRepository,
                     CalculadoraDescuentos calculadora) { ... }
```

Código 6.1: Inyección por constructor en `PedidoService`

Gracias a esto, `@InjectMocks` puede construir el servicio con cinco mocks sin modificar el código de producción.

6.6 Punto de vista de interacción

La figura 6.3 muestra el escenario principal (RF-05) del método `procesarPedido`.

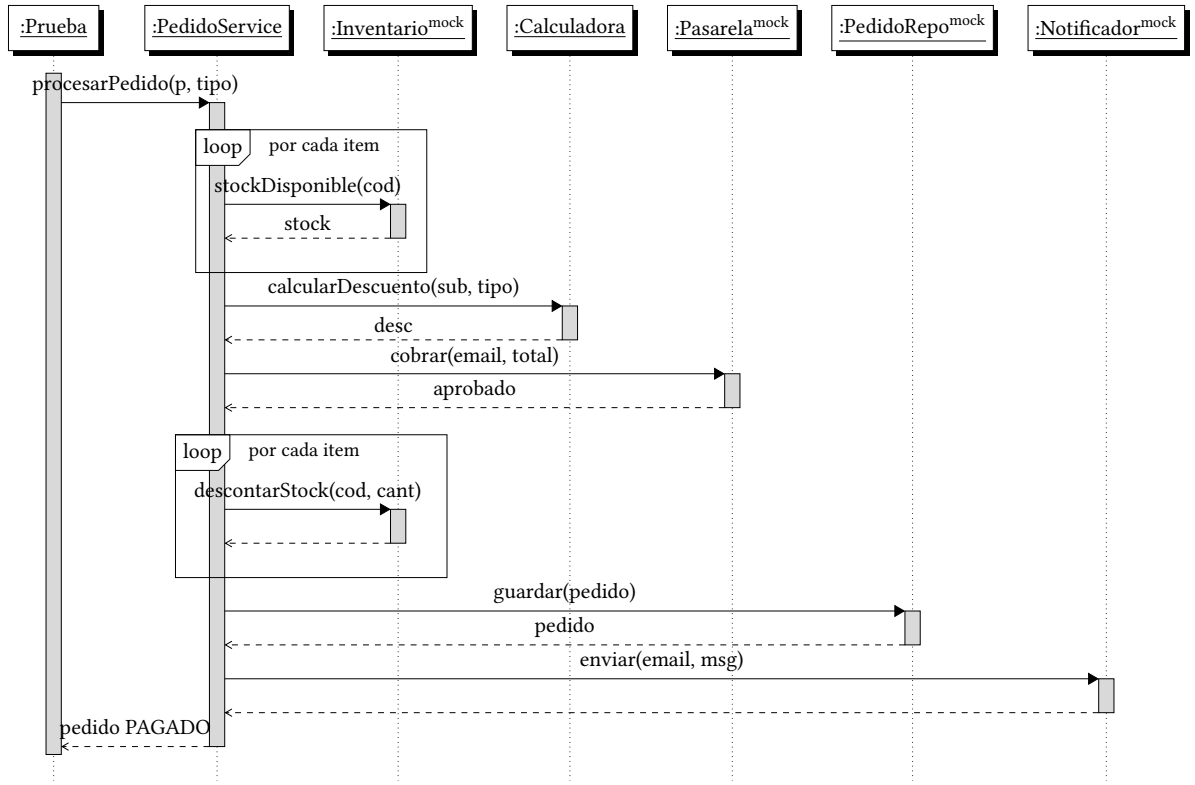


Figura 6.3: Diagrama de secuencia de `procesarPedido` (pago aprobado)

6.7 Punto de vista de estados

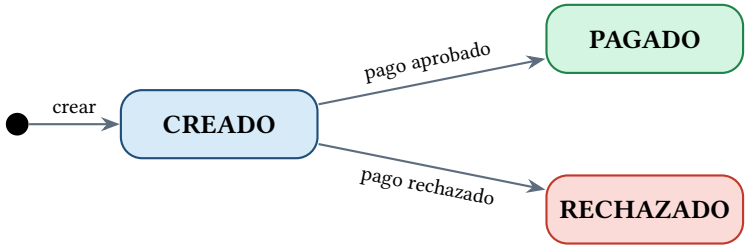


Figura 6.4: Diagrama de estados del pedido

6.8 Punto de vista de algoritmo

```
public Pedido procesarPedido(Pedido pedido, TipoCliente tipoCliente) {
    if (pedido == null || pedido.getItems().isEmpty()) {                // D1, D2
        throw new IllegalArgumentException("El pedido no tiene items");
    }
    double subtotal = 0;
    for (ItemPedido item : pedido.getItems()) {                         // D3
        String codigo = item.producto().codigo();
        if (inventario.stockDisponible(codigo) < item.cantidad()) {     // D4
            throw new StockInsuficienteException(codigo);
        }
        subtotal += item.subtotal();
    }
    double descuento = calculadora.calcularDescuento(subtotal, tipoCliente);
```

```
    double total = subtotal - descuento;

    ResultadoPago pago = pasarela.cobrar(pedido.getEmailCliente(), total);
    if (!pago.aprobado()) {                                          // D5
        pedido.setEstado(EstadoPedido.RECHAZADO);
        notificador.enviar(pedido.getEmailCliente(), "Pago rechazado: " + pago.mensaje());
        return pedido;
    }
    for (ItemPedido item : pedido.getItems()) {                      // D6
        inventario.descontarStock(item.producto().codigo(), item.cantidad());
    }
    pedido.setEstado(EstadoPedido.PAGADO);
    pedido.setTotal(total);
    Pedido guardado = pedidoRepository.guardar(pedido);
    notificador.enviar(pedido.getEmailCliente(), "Pedido " + pedido.getId()
            + " confirmado. Total: " + total + " Bs. Transacción: " + pago.codigoTransaccion());
    return guardado;
}
```

Código 6.2: Método `procesarPedido` con sus decisiones marcadas (D1–D6)

6.9 Diseño de las pruebas

Tabla 6.4: Estrategia de dobles por dependencia

Dependencia	Doble	Motivo
`InventarioRepository`	Mock	Evita la base de datos; permite fijar cualquier stock.
`PasarelaPago`	Mock	Evita cobros reales; permite simular aprobaciones y rechazos.
`Notificador`	Mock	Evita enviar correos; permite verificar el mensaje.
`PedidoRepository`	Mock	Evita la base de datos; `thenAnswer` devuelve el mismo pedido.
`CalculadoraDescuentos`	Real / Spy / Mock	Real en caja negra; Mock en caja blanca para aislar `procesarPedido`; Spy para verificar la llamada sin perder la lógica.
`GeneradorId` (estático)	`mockStatic`	Hace determinista el identificador aleatorio.

6.9.1 Matriz de trazabilidad

La matriz relaciona cada requisito de la SRS con el elemento de diseño y con los casos de prueba que lo verifican (capítulos 8 a 10):

Tabla 6.5: Trazabilidad SRS → diseño → casos de prueba

Requisito	Diseño	Casos de prueba
RF-01	`procesarPedido` (D2)	R1 (caja negra); CB-C2 (caja blanca)
(OBS-01)	`procesarPedido` (D1)	CB-C1: rama no especificada en la SRS
RF-02, RN-03	`procesarPedido` (D3, D4)	R2, L1 (caja negra); CB-C3, CB-C4 (caja blanca)
RF-03, RN-01/02/04	`calcularDescuento`	CD-01 a CD-15 (caja negra); K1 a K5 (caja blanca)
RF-04	`procesarPedido` (D5)	R3 (caja negra); CB-C5 (caja blanca); E06
RF-05	`procesarPedido` (D6)	R4 (caja negra); CB-C6, CB-C7 (caja blanca); Paso4, Paso5; E03, E07
RF-06	`obtenerPedido`	E02 (existente e inexistente)
RNF-02	–	Reporte JaCoCo: 100 % de ramas (capítulo 11)
RNF-03	puertos	Todas las pruebas: las dependencias externas son mocks
RNF-04	–	Suite completa en 1,3 s (capítulo 11)

6.10 Justificación de las decisiones de diseño

1. **Interfaces para lo externo**: permiten sustituir la implementación sin tocar el SUT y también cambiar de proveedor (otro banco) en producción.
2. **Inyección por constructor** en vez de `new` dentro del servicio: si `PedidoService` creara sus dependencias internamente, Mockito no podría reemplazarlas.
3. **Reglas de descuento en una clase pura**: `CalculadoraDescuentos` no tiene dependencias, así que se prueba sin mocks. En `PedidoService` se puede simular para aislar la orquestación.
4. **Objetos de valor como `record`**: son inmutables y fáciles de construir en las pruebas, por lo que no necesitan mocks.

Capítulo 7

Ejemplos prácticos de uso

El proyecto está en la carpeta `ejemplos-mockito/`. Los ejemplos del paquete `basico` recorren la API de menor a mayor complejidad.

7.1 Ciclo básico: crear, programar, ejecutar, verificar

```
InventarioRepository inventario = mock(InventarioRepository.class);   // 1. crear

when(inventario.stockDisponible("P-001")).thenReturn(25);             // 2. programar
when(inventario.stockDisponible("P-002")).thenReturn(3, 2, 1);        //    consecutivas

assertEquals(25, inventario.stockDisponible("P-001"));                // 3. ejecutar
assertEquals(0, inventario.stockDisponible("OTRO"));                  //    valor por defecto

inventario.descontarStock("P-001", 2);
verify(inventario).descontarStock("P-001", 2);                        // 4. verificar
verify(inventario, never()).descontarStock("P-999", 1);
```

Código 7.1: `E01_PrimerMockTest` (extracto)

7.2 Anotaciones e inyección

```
@ExtendWith(MockitoExtension.class)
class E02_AnotacionesTest {
    @Mock InventarioRepository inventario;
    @Mock PasarelaPago pasarela;
    @Mock Notificador notificador;
    @Mock PedidoRepository pedidoRepository;
    @Mock CalculadoraDescuentos calculadora;
    @InjectMocks PedidoService servicio;   // se construye con los 5 mocks

    @Test
    void obtenerPedidoInexistenteLanzaExcepcion() {
        when(pedidoRepository.buscarPorId("NO-EXISTE")).thenReturn(Optional.empty());
        assertThrows(PedidoNoEncontradoException.class,
                     () -> servicio.obtenerPedido("NO-EXISTE"));
    }
}
```

Código 7.2: `E02_AnotacionesTest` (extracto)

7.3 Argument matchers y ArgumentCaptor

```
when(inventario.stockDisponible(anyString())).thenReturn(10);
when(pasarela.cobrar(eq("luis@umss.edu.bo"), anyDouble()))
        .thenReturn(ResultadoPago.aprobado("TX-999"));
```

```
when(pedidoRepository.guardar(any(Pedido.class))).thenAnswer(inv -> inv.getArgument(0));

servicio.procesarPedido(pedido, TipoCliente.REGULAR);   // laptop de 600 Bs

verify(pasarela).cobrar(anyString(), doubleThat(monto -> monto == 540.0));
verify(notificador).enviar(eq("luis@umss.edu.bo"), contains("TX-999"));

verify(notificador).enviar(anyString(), mensajeCaptor.capture());
assertTrue(mensajeCaptor.getValue().startsWith("Pedido PED-7 confirmado"));
```

Código 7.3: `E03_ArgumentMatchersYCaptorTest` (extracto)

Lección aprendida

En la primera versión de este ejemplo usamos `argThat(monto -> monto == 540.0)`. Como el parámetro es un `double` primitivo, `argThat` devolvió `null` y Java lanzó un `NullPointerException` al convertirlo. Además, el *matcher* quedó "colgado" en la pila interna y provocó un `InvalidUseOfMatchersException` en otra prueba. La corrección fue usar `doubleThat`.

7.4 Excepciones y métodos void

```
when(inventario.stockDisponible("P-ERR"))
        .thenThrow(new IllegalStateException("Base de datos caída"));

doThrow(new RuntimeException("SMTP no disponible"))       // método void
        .when(notificador).enviar(anyString(), anyString());

doAnswer(inv -> { bandejaFalsa.add(inv.getArgument(0) + ": " + inv.getArgument(1));
                  return null; })
        .when(notificador).enviar(anyString(), anyString());
```

Código 7.4: `E04_ExcepcionesYVoidTest` (extracto)

7.5 Spies (mocks parciales)

```
CalculadoraDescuentos calculadora = spy(new CalculadoraDescuentos());
doReturn(99.0).when(calculadora).calcularDescuento(1000, TipoCliente.VIP);

assertEquals(99.0, calculadora.calcularDescuento(1000, TipoCliente.VIP));    // simulado
assertEquals(50.0, calculadora.calcularDescuento(500, TipoCliente.REGULAR)); // real
```

Código 7.5: `E05_SpyTest` (extracto)

7.6 Estilo BDD

```
// GIVEN
given(inventario.stockDisponible("TEC-01")).willReturn(5);
given(pasarela.cobrar(anyString(), anyDouble()))
        .willReturn(ResultadoPago.rechazado("Fondos insuficientes"));
// WHEN
Pedido resultado = servicio.procesarPedido(pedido, TipoCliente.VIP);
// THEN
```

```
assertThat(resultado.getEstado()).isEqualTo(EstadoPedido.RECHAZADO);
then(pasarela).should().cobrar("vip@umss.edu.bo", 270.0);   // 300 - 10 %
then(inventario).should(never()).descontarStock(anyString(), anyInt());
```

Código 7.6: `E06_BDDMockitoTest` (extracto)

7.7 Orden de llamadas y métodos estáticos

```
InOrder orden = inOrder(inventario, pasarela, pedidoRepository, notificador);
orden.verify(inventario).stockDisponible("MOU-01");
orden.verify(pasarela).cobrar("eva@umss.edu.bo", 50.0);   // se cobra ANTES de
orden.verify(inventario).descontarStock("MOU-01", 1);     // descontar el stock
orden.verify(pedidoRepository).guardar(pedido);
orden.verify(notificador).enviar(anyString(), anyString());

try (MockedStatic<GeneradorId> generador = mockStatic(GeneradorId.class)) {
    generador.when(GeneradorId::nuevoId).thenReturn("PED-FIJO");
    assertEquals("PED-FIJO", Pedido.crear("x@umss.edu.bo", List.of()).getId());
}   // al salir del bloque vuelve el comportamiento real
```

Código 7.7: `E07_OrdenYStaticTest` (extracto)

7.8 Strict stubs

```
when(inventario.stockDisponible("A")).thenReturn(1);
lenient().when(inventario.stockDisponible("B")).thenReturn(2); // sin lenient() -> falla
assertEquals(1, inventario.stockDisponible("A"));
```

Código 7.8: `E08_StrictStubsTest` (extracto)

Capítulo 8

Cómo se diseña y se escribe un caso de prueba, paso a paso

Los capítulos anteriores mostraron *qué* hace Mockito. Este capítulo muestra **cómo trabajamos**: el procedimiento que seguimos para pasar de un requisito (o de un código) a un caso de prueba y, de ahí, a una prueba automatizada que pasa. Se explica cada decisión y se muestran también los **errores reales** que aparecen por el camino, porque así se aprende a usar la herramienta.

8.1 El proceso general

Seguimos el proceso de diseño de pruebas de la norma **ISO/IEC/IEEE 29119-4** (*Test techniques*), adaptado a pruebas unitarias con Mockito:

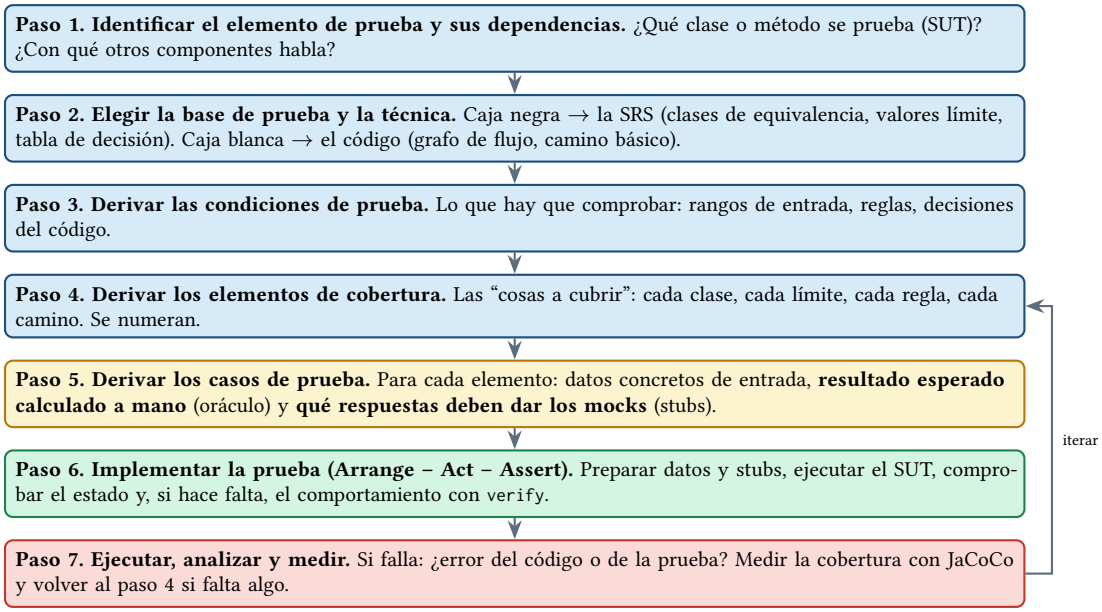


Figura 8.1: Proceso que seguimos para cada caso de prueba

8.2 La ficha de un caso de prueba

Cada caso de prueba se documenta con una **ficha**, basada en la especificación de casos de prueba de IEEE 829 / ISO/IEC/IEEE 29119-3. La ficha se escribe **antes** del código: el código solo traduce la ficha.

Tabla 8.1: Campos de la ficha de caso de prueba

Campo	Qué contiene
ID y nombre	Identificador único (p. ej., CB-05) y nombre descriptivo.
Origen / técnica	De dónde sale: requisito de la SRS o camino del código, y la técnica usada.
Precondiciones	Estado necesario antes de ejecutar.
Datos de entrada	Valores concretos que recibe el SUT.
Configuración de dobles	Qué responde cada mock (*stubs*). Es lo propio de Mockito.
Pasos	Acciones que realiza la prueba.
Resultado esperado	Salida calculada **a mano** a partir de la SRS (oráculo).
Verificaciones	Qué se comprueba: estado (`assert`) y comportamiento (`verify`).
Resultado obtenido	Lo que produjo la ejecución, y el veredicto (APROBADO / FALLIDO).

8.3 Paso 1 aplicado: ¿qué se simula y qué no?

Antes de escribir una sola línea hay que decidir qué dependencias del SUT se reemplazan por dobles. Usamos este criterio:

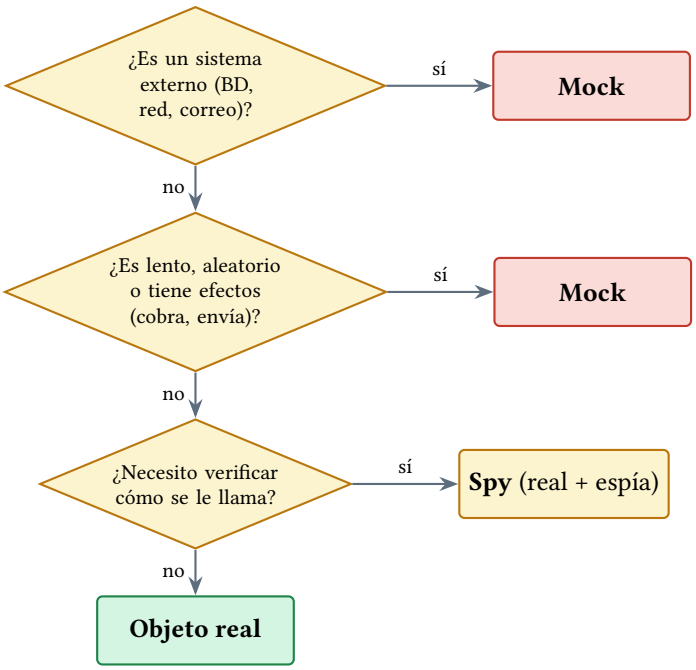


Figura 8.2: Criterio para decidir el tipo de doble

Tabla 8.2: Decisión aplicada a las dependencias de `PedidoService`

Dependencia	Decisión	Razonamiento
`InventarioRepository`	Mock	Externa (base de datos) → mock. Además, necesitamos fijar el stock a voluntad.
`PasarelaPago`	Mock	Externa y con efectos (cobra dinero) → mock. Permite simular "rechazado".
`Notificador`	Mock	Externa y con efectos (envía correos) → mock.
`PedidoRepository`	Mock	Externa (base de datos) → mock.
`CalculadoraDescuentos`	Spy / real	Lógica propia, rápida y sin efectos → real. Se usa `@Spy` para poder verificar la llamada. En caja blanca se simula para *aislar* `procesarPedido` (ver capítulo 10).
`Pedido`, `Producto`...	Reales	Objetos de valor: se crean con `new`, nunca se simulan.

8.4 Paso 6 aplicado: construir una prueba de rojo a verde

Se toma el criterio de aceptación de **RF-05**: *"Dado un pedido de 2 monitores de 250 Bs para un cliente REGULAR y un banco que aprueba, cuando se procesa, entonces se cobran 450 Bs, el estado es PAGADO..."*. A continuación se muestra cómo la construimos, **versión por versión**. Cada versión está en el paquete `pasoapaso` del proyecto. Las versiones en rojo están desactivadas con `@Disabled` y se ejecutan con `./run.sh rojos`.

8.4.1 Versión 1: la estructura

```java
@ExtendWith(MockitoExtension.class)            // 1.1 activar Mockito
class Paso1_EstructuraTest {
    @Mock InventarioRepository inventario;     // 1.2 un @Mock por dependencia externa
    @Mock PasarelaPago pasarela;
    @Mock Notificador notificador;
    @Mock PedidoRepository pedidoRepository;
    @Spy  CalculadoraDescuentos calculadora;   // 1.3 lógica propia: real
    @InjectMocks PedidoService servicio;       // 1.4 el SUT, construido con los dobles

    @Test
    void elServicioSeConstruyeConSusDobles() {
        assertNotNull(servicio);               // 1.5 primera comprobación mínima
    }
}
```

Código 8.1: `Paso1_EstructuraTest`: preparar el escenario

Qué hicimos: escribimos la clase de prueba con un campo por dependencia, según la tabla del paso 1. **Resultado:** verde. Esto confirma que `@InjectMocks` encontró el constructor de 5 parámetros.

8.4.2 Versión 2: Arrange – Act – Assert sin programar los mocks (rojo)

```java
// Arrange: solo los datos de entrada
Producto monitor = new Producto("MON-01", "Monitor", 250.0);
Pedido pedido = new Pedido("PED-1", "cliente@umss.edu.bo", List.of(new ItemPedido(monitor, 2)));
// Act
Pedido resultado = servicio.procesarPedido(pedido, TipoCliente.REGULAR);
// Assert
assertEquals(EstadoPedido.PAGADO, resultado.getEstado());
```

Código 8.2: `Paso2_RojoSinStubsTest`

```
pedidoValidoQuedaPagado() ✘ Stock insuficiente para el producto MON-01
  => bo.edu.umss.pedidos.modelo.StockInsuficienteException: Stock insuficiente para el producto
     MON-01
      bo.edu.umss.pedidos.servicio.PedidoService.procesarPedido(PedidoService.java:50)
```

Código 8.3: Salida real de la ejecución

Análisis: el servicio lanzó "stock insuficiente" aunque nosotros no pensábamos en el stock. La causa es que un mock **sin programar** devuelve el valor por defecto: `stockDisponible()` devolvió `0`, y $0 < 2$. **Lección:** hay que programar (*stub*) cada consulta que el camino hace a un mock.

8.4.3 Versión 3: programar solo el stock (rojo otra vez)

```
when(inventario.stockDisponible("MON-01")).thenReturn(10);      // NUEVO: hay stock
```

Código 8.4: `Paso3_RojoStubIncompletoTest`: se añade una línea

```
pedidoValidoQuedaPagado() ✘ Cannot invoke "bo.edu.umss.pedidos.modelo.ResultadoPago.aprobado()"
                             because "<local9>" is null
     bo.edu.umss.pedidos.servicio.PedidoService.procesarPedido(PedidoService.java:59)
```

Código 8.5: Salida real de la ejecución

Análisis: ahora el código avanzó hasta la línea 59 (`if (!pago.aprobado())`). `pasarela.cobrar(...)` no estaba programado y devolvió `null`, el valor por defecto para objetos; al llamar a `.aprobado()` sobre `null` se produjo un `NullPointerException`. `<local9>` es la variable local `pago`. **Lección:** hay que *seguir el camino del código* y programar **todas** las dependencias que ese camino consulta, no solo la primera.

8.4.4 Versión 4: todos los stubs del camino (verde)

Antes de escribir el stub del cobro hay que saber **cuánto** se va a cobrar. Lo calculamos **a mano**, desde la SRS, sin mirar el código:

$$\begin{aligned} \text{Subtotal} &= 2 \times 250 = 500 \text{ Bs} \\ \text{Tramo (RN-01)} &= 500 \geq 500 \Rightarrow 10\ \%\text{; cliente REGULAR} \Rightarrow \text{sin adicional (RN-02)} \\ \text{Descuento} &= 500 \times 0{,}10 = 50 \text{ Bs} \\ \text{Total (RN-04)} &= 500 - 50 = \mathbf{450} \text{ Bs} \end{aligned}$$

```
when(inventario.stockDisponible("MON-01")).thenReturn(10);
when(pasarela.cobrar("cliente@umss.edu.bo", 450.0))                          // NUEVO
        .thenReturn(ResultadoPago.aprobado("TX-1"));
when(pedidoRepository.guardar(any(Pedido.class))).thenAnswer(inv -> inv.getArgument(0)); // NUEVO

Pedido resultado = servicio.procesarPedido(pedido, TipoCliente.REGULAR);

assertEquals(EstadoPedido.PAGADO, resultado.getEstado());
assertEquals(450.0, resultado.getTotal(), 0.001);
```

Código 8.6: `Paso4_VerdeTest`

Decisiones:

- El stub del cobro usa el valor **exacto** `450.0`, no `anyDouble()`. Si el servicio cobrara otro monto, el stub no coincidiría y la prueba fallaría. Así, el stub también *verifica* el cálculo.
- El repositorio usa `thenAnswer(inv -> inv.getArgument(0))`: "devuelve el mismo pedido que te pasen". Así se imita a una base de datos real.

- La tolerancia `0.001` en `assertEquals` evita falsos fallos por la representación de los `double`.

Resultado: verde.

8.4.5 Versión 5: verificar el comportamiento

El estado PAGADO no garantiza que se haya descontado el stock ni enviado el correo. RF-05 lo exige, así que se añaden verificaciones de comportamiento:

```
verify(inventario).descontarStock("MON-01", 2);                                    // NUEVO
verify(pedidoRepository).guardar(pedido);                                          // NUEVO
verify(notificador).enviar(eq("cliente@umss.edu.bo"), startsWith("Pedido PED-1 confirmado")); //
    NUEVO
```

Código 8.7: `Paso5_VerificarComportamientoTest`: se añaden 3 líneas

⚠ Error real que cometimos al escribir esta versión

La primera vez escribimos `enviar("cliente@umss.edu.bo", startsWith("Pedido PED-1 confirmado"))`, mezclando un valor literal con un *matcher*. Mockito respondió con `InvalidUseOfMatchersException`: cuando se usa un matcher, **todos** los argumentos deben serlo. La corrección fue envolver el correo con `eq(...)`. El capítulo 4 explica el motivo: la pila de matchers tenía 1 elemento para 2 argumentos.

8.4.6 Versión 6: ¿qué pasa si la prueba está mal? (rojo)

Para comprobar que la prueba detecta errores, simulamos un error del tester: olvidar el descuento y esperar un cobro de 500 Bs.

```
verify(pasarela).cobrar("cliente@umss.edu.bo", 500.0);   // ERROR del tester: faltó el descuento
```

Código 8.8: `Paso6_RojoVerificacionTest`

```
Wanted but not invoked:
pasarela.cobrar(
    "cliente@umss.edu.bo",
    500.0d
);
-> at ...Paso6_RojoVerificacionTest.seCobraElSubtotalSinDescuento(Paso6_RojoVerificacionTest.java
    :57)

However, there was exactly 1 interaction with this mock:
pasarela.cobrar(
    "cliente@umss.edu.bo",
    450.0d
);
-> at bo.edu.umss.pedidos.servicio.PedidoService.procesarPedido(PedidoService.java:58)
```

Código 8.9: Salida real de la ejecución

Cómo se lee: Mockito dice qué esperábamos (*Wanted*: cobrar 500), dónde lo pedimos (línea 57 de la prueba) y qué pasó en realidad (*However*: se cobró 450 en la línea 58 del servicio). Con eso el tester sabe de inmediato que el error está en su expectativa, no en el código.

8.4.7 Resumen: lista de control para escribir cualquier prueba

1. Escribir la **ficha** del caso (entradas, oráculo, stubs).

2. Crear la clase con `@ExtendWith`, los `@Mock` y el `@InjectMocks`.
3. **Arrange**: crear los datos; programar un stub por cada consulta que el camino hace a un mock. Con `STRICT_STUBS`, ni uno más.
4. **Act**: una sola llamada al SUT.
5. **Assert**: primero el estado (`assertEquals`), luego el comportamiento necesario (`verify`, `never`).
6. Ejecutar. Si falla, leer el mensaje: ¿error del código o de la prueba?
7. Comprobar que la prueba **puede fallar** (versión 6): una prueba que nunca falla no prueba nada.

Capítulo 9

Pruebas de caja negra, paso a paso

En caja negra **no se mira el código**. Los casos se obtienen solo de la SRS (capítulo 5). Se aplican las técnicas a dos elementos: `CalculadoraDescuentos` (clases de equivalencia y valores límite) y `PedidoService.procesarPedido` (tabla de decisión).

9.1 `CalculadoraDescuentos`: clases de equivalencia y valores límite

9.1.1 Paso 1: elemento de prueba y dependencias

El elemento es el método `calcularDescuento(subtotal, tipo)`. **No tiene dependencias**, así que **no se necesitan mocks**. Esto también es una decisión: no se usa Mockito donde no hace falta.

9.1.2 Paso 2: base de prueba

Se toman de la SRS el requisito RF-03 y las reglas RN-01, RN-02 y RN-04:

> *RN-01: menor que 100 Bs → 0 %; de 100 a 499,99 Bs → 5 %; 500 Bs o más → 10 %. Un subtotal negativo es inválido. RN-02: un cliente VIP recibe 5 puntos adicionales. RN-04: redondeo a 2 decimales.*

9.1.3 Paso 3: condiciones de prueba (entradas y salidas)

Tabla 9.1: Análisis de entradas y salidas

Variable	Tipo	Dominio según la SRS
`subtotal` (entrada)	número real, Bs	$(-\infty, +\infty)$, con 4 zonas: negativo, $[0, 100)$, $[100, 500)$, $[500, +\infty)$
`tipo` (entrada)	enumeración	{REGULAR, VIP}
descuento (salida)	número real, Bs	≥ 0, 2 decimales; o un error si la entrada es inválida

9.1.4 Paso 4a: clases de equivalencia

Una **clase de equivalencia** es un conjunto de valores que, según la especificación, el sistema debe tratar *igual*. Si el sistema funciona con un valor de la clase, se supone que funciona con todos, así que basta con probar uno. Se aplican las reglas de Myers:

- Si la condición es un **rango**, se crea una clase válida por cada tramo y clases inválidas para lo que queda fuera.
- Si la condición es un **conjunto de valores** tratados de forma distinta, se crea una clase por valor.

Tabla 9.2: Clases de equivalencia derivadas

Clase	Variable	Condición	Tipo	De dónde sale
CE1	subtotal	$s < 0$	inválida	RN-01: "negativo es inválido"
CE2	subtotal	$0 \leq s < 100$	válida	RN-01: tramo 0 %
CE3	subtotal	$100 \leq s < 500$	válida	RN-01: tramo 5 %
CE4	subtotal	$s \geq 500$	válida	RN-01: tramo 10 %
CE5	tipo	REGULAR	válida	RN-02: sin adicional
CE6	tipo	VIP	válida	RN-02: +5 %

9.1.5 Paso 4b: valores límite

Los errores de programación se concentran en los bordes (escribir > en lugar de >=). Por eso, para cada frontera b entre clases se prueban el **propio límite** y el valor **inmediatamente anterior**. Como los montos tienen 2 decimales (RN-04), "inmediatamente anterior" es $b - 0{,}01$.

Tabla 9.3: Derivación de los valores límite

Frontera	Entre	Valor anterior ($b - 0{,}01$)	Límite (b)
0	CE1 | CE2	$-0{,}01$ (CE1)	0 (CE2)
100	CE2 | CE3	99,99 (CE2)	100 (CE3)
500	CE3 | CE4	499,99 (CE3)	500 (CE4)

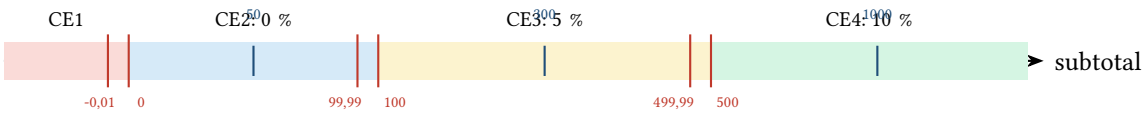


Figura 9.1: Particiones, valores límite (rojo) y representantes (azul)

9.1.6 Paso 4c: combinación de las dos variables

Hay 4 clases de subtotal y 2 de tipo. Decidimos así cómo combinarlas:

- **Con REGULAR** (CE5): todas las clases válidas de subtotal, con su representante y sus límites (8 casos).
- **Con VIP** (CE6): la regla RN-02 es *aditiva* e igual en todos los tramos, así que basta cubrir cada clase válida de subtotal al menos una vez con VIP, usando su límite inferior (4 casos: 50, 100, 500 y un representante alto, 2000).
- **Clase inválida** (CE1): siguiendo a Myers, **un caso inválido cubre una sola clase inválida** y se combina con valores válidos del resto (REGULAR). Se prueban el límite ($-0{,}01$) y un representante (-100).

9.1.7 Paso 5: oráculo y tabla de casos

El resultado esperado de cada caso se calcula **a mano** con las reglas de la SRS:

Tabla 9.4: Casos de prueba de caja negra para `CalculadoraDescuentos`

ID	Clases	Tipo de valor	Subtotal	Tipo	Cálculo del oráculo	Esperado
CD-01	CE2, CE5	representante	50	REG	50×0	0,00
CD-02	CE2, CE5	límite	0	REG	0×0	0,00
CD-03	CE2, CE5	anterior a 100	99,99	REG	$99{,}99 \times 0$	0,00
CD-04	CE3, CE5	límite	100	REG	$100 \times 0{,}05$	5,00
CD-05	CE3, CE5	representante	300	REG	$300 \times 0{,}05$	15,00
CD-06	CE3, CE5	anterior a 500	499,99	REG	$499{,}99 \times 0{,}05 = 24{,}9995 \approx$	25,00
CD-07	CE4, CE5	límite	500	REG	$500 \times 0{,}10$	50,00
CD-08	CE4, CE5	representante	1000	REG	$1000 \times 0{,}10$	100,00
CD-09	CE2, CE6	representante	50	VIP	$50 \times (0 + 0{,}05)$	2,50
CD-10	CE3, CE6	límite	100	VIP	$100 \times (0{,}05 + 0{,}05)$	10,00
CD-11	CE4, CE6	límite	500	VIP	$500 \times (0{,}10 + 0{,}05)$	75,00
CD-12	CE4, CE6	representante	2000	VIP	$2000 \times 0{,}15$	300,00
CD-13	CE1	anterior a 0	$-0{,}01$	REG	RN-01: inválido	error
CD-14	CE1	representante	-100	REG	RN-01: inválido	error
CD-15	CE5, CE6	propiedad	varios	ambos	RN-02: VIP $>$ REGULAR	verdadero

Detalle del caso CD-06

$499{,}99 \times 0{,}05 = 24{,}9995$. RN-04 exige 2 decimales, así que el valor esperado es 25,00. Este caso comprueba, además del límite, la **regla de redondeo**: si el sistema truncara en lugar de redondear, daría 24,99 y la prueba fallaría.

Comprobación de cobertura de clases. Antes de programar se verifica que cada clase tenga al menos un caso:

Tabla 9.5: Matriz clases × casos

	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15
CE1													X	X	
CE2	X	X	X						X						
CE3				X	X	X				X					
CE4							X	X			X	X			
CE5	X	X	X	X	X	X	X	X					X	X	X
CE6									X	X	X	X			X

9.1.8 Paso 6: de la tabla al código, paso a paso

6.1 Primer caso como prueba simple. Se empieza traduciendo un solo caso (CD-07), para validar la idea:

```
class CalculadoraDescuentosCajaNegraTest {
    private final CalculadoraDescuentos calculadora = new CalculadoraDescuentos(); // objeto REAL

    @Test
    void cd07LimiteInferiorCE4() {
        assertEquals(50.00, calculadora.calcularDescuento(500, TipoCliente.REGULAR), 0.001);
    }
}
```

Código 9.1: Un caso aislado

6.2 Se generaliza a una prueba parametrizada. Los 12 casos válidos tienen la misma forma (entrada → número esperado), así que en lugar de 12 métodos casi iguales se usa `@ParameterizedTest`:

cada fila del `@CsvSource` es una fila de la tabla.

```
@ParameterizedTest(name = "[{index}] {0}: subtotal={1}, {2} -> descuento={3}")
@CsvSource({
    "CE2 representante,       50,     REGULAR, 0.00",    // CD-01
    "CE2 límite inferior,     0,      REGULAR, 0.00",    // CD-02
    "CE2 límite superior,     99.99,  REGULAR, 0.00",    // CD-03
    "CE3 límite inferior,     100,    REGULAR, 5.00",    // CD-04
    "CE3 representante,       300,    REGULAR, 15.00",   // CD-05
    "CE3 límite superior,     499.99, REGULAR, 25.00",   // CD-06
    "CE4 límite inferior,     500,    REGULAR, 50.00",   // CD-07
    "CE4 representante,       1000,   REGULAR, 100.00",  // CD-08
    "CE2 VIP,                 50,     VIP,     2.50",    // CD-09
    "CE3 VIP límite inferior, 100,    VIP,     10.00",   // CD-10
    "CE4 VIP límite inferior, 500,    VIP,     75.00",   // CD-11
    "CE4 VIP representante,   2000,   VIP,     300.00"   // CD-12
})
void clasesValidasYValoresLimite(String caso, double subtotal, TipoCliente tipo, double esperado)
     {
    assertEquals(esperado, calculadora.calcularDescuento(subtotal, tipo), 0.001, caso);
}
```

Código 9.2: Clases válidas y límites (`CalculadoraDescuentosCajaNegraTest`)

6.3 La clase inválida. El resultado esperado no es un número sino un error, así que se usa `assertThrows`:

```
@ParameterizedTest(name = "CE1 inválida: subtotal={0}")
@CsvSource({"-0.01", "-100"})
void claseInvalidaSubtotalNegativo(double subtotal) {
    assertThrows(IllegalArgumentException.class,
            () -> calculadora.calcularDescuento(subtotal, TipoCliente.REGULAR));
}
```

Código 9.3: Clase inválida CE1 (CD-13 y CD-14)

9.1.9 Paso 7: ejecución

```
Caja negra - CalculadoraDescuentos ✔
|- Un VIP siempre recibe más descuento que un REGULAR ✔          (CD-15)
|- claseInvalidaSubtotalNegativo(double) ✔
|  |- CE1 inválida: subtotal="-0.01" ✔                            (CD-13)
|  '- CE1 inválida: subtotal="-100" ✔                             (CD-14)
'- clasesValidasYValoresLimite(String, double, TipoCliente, double) ✔
   |- [1] "CE2 representante": subtotal="50", "REGULAR" -> descuento="0.00" ✔
   |- [3] "CE2 límite superior": subtotal="99.99", "REGULAR" -> descuento="0.00" ✔
   |- [6] "CE3 límite superior": subtotal="499.99", "REGULAR" -> descuento="25.00" ✔
   |- [7] "CE4 límite inferior": subtotal="500", "REGULAR" -> descuento="50.00" ✔
   '- ... (12 filas en total)
```

Código 9.4: Salida de la ejecución (extracto; los ID entre paréntesis son anotaciones nuestras)

9.2 `PedidoService.procesarPedido`: tabla de decisión

9.2.1 Pasos 1 y 2: elemento, dependencias y base de prueba

El elemento es `procesarPedido`. Tiene 5 dependencias, que se resuelven con la decisión del capítulo 8: cuatro mocks y la calculadora real. La base de prueba son los requisitos RF-01, RF-02, RF-04 y RF-05 y el caso de uso CU-01 con sus flujos alternativos FA-1, FA-2 y FA-3.

9.2.2 Paso 3: condiciones y acciones

Leyendo el caso de uso CU-01 se identifican las **condiciones** que cambian el resultado y las **acciones** posibles:

Tabla 9.6: Condiciones y acciones extraídas de la SRS

	ID	Descripción (origen)
Condiciones	C1	¿El pedido tiene items? (RF-01, paso 2 / FA-1)
	C2	¿Hay stock suficiente para todos los items? (RF-02, paso 3 / FA-2)
	C3	¿La pasarela aprueba el pago? (RF-04/05, paso 6 / FA-3)
Acciones	A1	Error “el pedido no tiene items”
	A2	Error “stock insuficiente”
	A3	Estado RECHAZADO + notificar motivo
	A4	Estado PAGADO + descontar stock + registrar + notificar confirmación

9.2.3 Paso 4a: tabla de decisión completa

Con 3 condiciones binarias hay $2^3 = 8$ combinaciones. Se escriben **todas** y, para cada una, se determina con la SRS qué acción corresponde:

Tabla 9.7: Tabla de decisión completa (8 reglas)

	1	2	3	4	5	6	7	8
C1 ¿tiene items?	N	N	N	N	S	S	S	S
C2 ¿hay stock?	N	N	S	S	N	N	S	S
C3 ¿pago aprobado?	N	S	N	S	N	S	N	S
A1 error sin items	X	X	X	X				
A2 error sin stock					X	X		
A3 RECHAZADO							X	
A4 PAGADO								X

9.2.4 Paso 4b: simplificar la tabla

Se buscan columnas con la **misma acción** que solo difieren en condiciones que **no influyen**:

- Reglas 1–4: si no hay items (C1 = N), el proceso termina en el paso 2 del CU-01, así que C2 y C3 no importan ⇒ se fusionan en **R1** con “–” (indiferente).
- Reglas 5–6: sin stock, el proceso termina en el paso 3 y nunca se cobra, así que C3 no importa ⇒ **R2**.
- Las reglas 7 y 8 quedan como **R3** y **R4**.

Tabla 9.8: Tabla de decisión simplificada (4 reglas = 4 casos)

	R1	**R2**	**R3**	**R4**
C1 ¿tiene items?	N	S	S	S
C2 ¿hay stock?	–	N	S	S
C3 ¿pago aprobado?	–	–	N	S
Acción	A1	A2	A3	A4
Reglas originales	1–4	5–6	7	8

9.2.5 Paso 4c: valor límite de la condición C2

La condición C2 esconde una comparación numérica (RN-03: suficiente si *stock* $\geq$ *cantidad*). Se aplica también análisis de valores límite: `stock = cantidad` (límite, debe aprobarse; caso **L1**) y `stock = cantidad − 1` (debe fallar; lo cubre R2 con stock 1 y cantidad 2).

9.2.6 Paso 5: de cada regla a datos concretos y stubs

Aquí entra Mockito. Para cada regla hay que decidir **qué debe responder cada mock** para que se cumplan las condiciones. La clave es que **C2 y C3 no dependen del pedido, sino de lo que respondan los sistemas externos**: sin mocks no podríamos elegirlas.

Tabla 9.9: Condición → cómo se controla

Condición	**Cómo la controlamos en la prueba**
C1	Con los *datos*: pedido con `List.of()` (N) o con un item (S).
C2	Con un *stub*: `when(inventario.stockDisponible("MON-01")).thenReturn(1)` (N) o `thenReturn(10)` (S).
C3	Con un *stub*: `when(pasarela.cobrar(...)).thenReturn(rechazado(...))` (N) o `aprobado(...)` (S).

¿Por qué no se usa `verify` en caja negra?

En caja negra solo se observan las **salidas** que promete la SRS: el estado del pedido, el total o la excepción. Los mocks se usan solo como **stubs** que *proveen entradas*. Si verificáramos llamadas internas, la prueba dependería de la implementación y dejaría de ser de caja negra.

Con `STRICT_STUBS`, además, solo se programa lo que el camino realmente usa. Por ejemplo, en R2 **no** se programa la pasarela, porque la SRS dice que sin stock no se cobra. Si la programáramos, Mockito avisaría de un stub innecesario.

9.2.7 Fichas de los casos

El pedido de referencia es `pedidoDe(n)` = 1 item de `MON-01` (Monitor, 250 Bs) $\times n$ para `cliente@umss.edu.bo`.

R1 – Pedido sin items es rechazado

Origen / técnica	Caja negra, tabla de decisión, regla R1. RF-01, CU-01 FA-1.
Precondiciones	Servicio construido con 4 mocks y la calculadora real.
Datos de entrada	Pedido `PED-0` con `List.of()`; tipo REGULAR.
Dobles (stubs)	Ninguno (el proceso termina antes de consultar a nadie).
Pasos	Llamar a `procesarPedido`.
Resultado esperado	`IllegalArgumentException`.
Verificaciones	`assertThrows(IllegalArgumentException.class, ...)`.
Resultado obtenido	Coincide con lo esperado. **APROBADO** ✔

R2 – Sin stock suficiente

Origen / técnica	Caja negra, regla R2 (y valor límite stock = cantidad − 1). RF-02, FA-2.
Precondiciones	Igual que R1.
Datos de entrada	`pedidoDe(2)`; tipo REGULAR.
Dobles (stubs)	`inventario.stockDisponible("MON-01")` → `1`.
Pasos	Llamar a `procesarPedido`.
Resultado esperado	`StockInsuficienteException`.
Verificaciones	`assertThrows(StockInsuficienteException.class, ...)`.
Resultado obtenido	Coincide con lo esperado. **APROBADO** ✔

R3 – Pago rechazado

Origen / técnica	Caja negra, regla R3. RF-04, FA-3.
Precondiciones	Igual que R1.
Datos de entrada	`pedidoDe(2)`; tipo REGULAR.
Dobles (stubs)	`stockDisponible("MON-01")` → `10`; `cobrar(anyString(), anyDouble())` → `rechazado("Tarjeta vencida")`.
Pasos	Llamar a `procesarPedido`.
Resultado esperado	Pedido en estado `RECHAZADO`.
Verificaciones	`assertEquals(RECHAZADO, resultado.getEstado())`.
Resultado obtenido	Coincide con lo esperado. **APROBADO** ✔

R4 – Pago aprobado

Origen / técnica	Caja negra, regla R4. RF-05, flujo básico CU-01.
Precondiciones	Igual que R1.
Datos de entrada	`pedidoDe(2)`: subtotal 500; tipo REGULAR.
Dobles (stubs)	`stockDisponible` → `10`; `cobrar(...)` → `aprobado("TX-1")`; `guardar(any())` → el mismo pedido.
Pasos	Llamar a `procesarPedido`.
Resultado esperado	Estado `PAGADO`; total $= 500 - 10\ \%= 450{,}00$.
Verificaciones	`assertEquals(PAGADO, ...)`; `assertEquals(450.0, total, 0.001)`.
Resultado obtenido	Coincide con lo esperado. **APROBADO** ✔

L1 – Valor límite: stock igual a la cantidad

Origen / técnica	Caja negra, valor límite de C2. RN-03 (“$\geq$”).
Precondiciones	Igual que R1.
Datos de entrada	`pedidoDe(2)`; tipo REGULAR.
Dobles (stubs)	`stockDisponible` → `2`; `cobrar` → `aprobado("TX-2")`; `guardar` → el mismo pedido.
Pasos	Llamar a `procesarPedido`.
Resultado esperado	Estado `PAGADO` (el stock justo alcanza).
Verificaciones	`assertEquals(PAGADO, ...)`.
Resultado obtenido	Coincide con lo esperado. **APROBADO** ✔

9.2.8 Paso 6: código de un caso, línea por línea

```
@Test
@DisplayName("R4: pago aprobado -> PAGADO y total = 500 - 10 % = 450")
void r4PagoAprobado() {
    // ARRANGE -- condición C2 = S: el inventario "dice" que hay 10 unidades
    when(inventario.stockDisponible("MON-01")).thenReturn(10);
    // ARRANGE -- condición C3 = S: el banco "aprueba" cualquier cobro
    when(pasarela.cobrar(anyString(), anyDouble())).thenReturn(ResultadoPago.aprobado("TX-1"));
    // ARRANGE -- el registro devuelve el mismo pedido que recibe (como una BD real)
    when(pedidoRepository.guardar(any())).thenAnswer(inv -> inv.getArgument(0));

    // ACT -- condición C1 = S: pedido con 1 item (2 monitores de 250 Bs)
```

```
    Pedido resultado = servicio.procesarPedido(pedidoDe(2), TipoCliente.REGULAR);

    // ASSERT -- solo salidas observables prometidas por RF-05
    assertEquals(EstadoPedido.PAGADO, resultado.getEstado());
    assertEquals(450.0, resultado.getTotal(), 0.001);
}
```

Código 9.5: Regla R4 comentada paso a paso

9.2.9 Paso 7: ejecución

```
Caja negra - PedidoService (tabla de decisión) ✔
|- R1: pedido sin items es rechazado ✔
|- R2: sin stock suficiente se lanza StockInsuficienteException ✔
|- R3: pago rechazado deja el pedido en estado RECHAZADO ✔
|- R4: pago aprobado -> PAGADO y total = 500 - 10 % = 450 ✔
`- Valor límite de stock: stock == cantidad es suficiente ✔
```

Código 9.6: Salida de la ejecución (ordenada)

¿Detectan errores estas pruebas?

Lo comprobamos introduciendo un error a propósito en el código (*mutación*): cambiar `stock < cantidad` por `stock <= cantidad`. Al ejecutar, **falló el caso L1** (valor límite), junto con 3 pruebas de caja blanca y de ejemplos. Sin el caso de valor límite, ese error habría pasado inadvertido en la caja negra.

Capítulo 10

Pruebas de caja blanca, paso a paso

En caja blanca la base de prueba es el **código fuente**. La técnica es el **camino básico de McCabe**: se construye el grafo de flujo, se calcula la complejidad ciclomática $V(G)$ y se diseña un caso por cada camino independiente. Se aplica a `procesarPedido` (código 6.2) y a `calcularDescuento`.

10.1 `procesarPedido`: camino básico

10.1.1 Paso 1: numerar el código y marcar las decisiones

Se parte del código 6.2 (sección del SDD), cuyas líneas están numeradas. Se marcan las **decisiones**: cada `if`, cada condición de un `for` y cada condición simple de una condición compuesta.

Tabla 10.1: Decisiones encontradas en el código

ID	Línea	Decisión y cómo se tomó
D1	2	`pedido == null`. Primera parte del `||`.
D2	2	`pedido.getItems().isEmpty()`. Segunda parte del `||`. *Una condición compuesta se separa en condiciones simples*, porque cada una es una bifurcación distinta.
D3	6	`for`: ¿queda otro item por validar?
D4	8	`stockDisponible(codigo) < item.cantidad()`.
D5	17	`!pago.aprobado()`.
D6	22	`for`: ¿queda otro item por descontar?

10.1.2 Paso 2: agrupar las líneas en nodos

Para construir los nodos del grafo se aplican tres reglas:

1. Cada **decisión** es un nodo (nodo predicado).
2. Las sentencias **secuenciales consecutivas**, sin decisiones entre medio, se agrupan en un único nodo.
3. Se añade un **nodo de salida** único, al que llegan todos los `return` y `throw`.

Tabla 10.2: Correspondencia nodo ↔ líneas de código

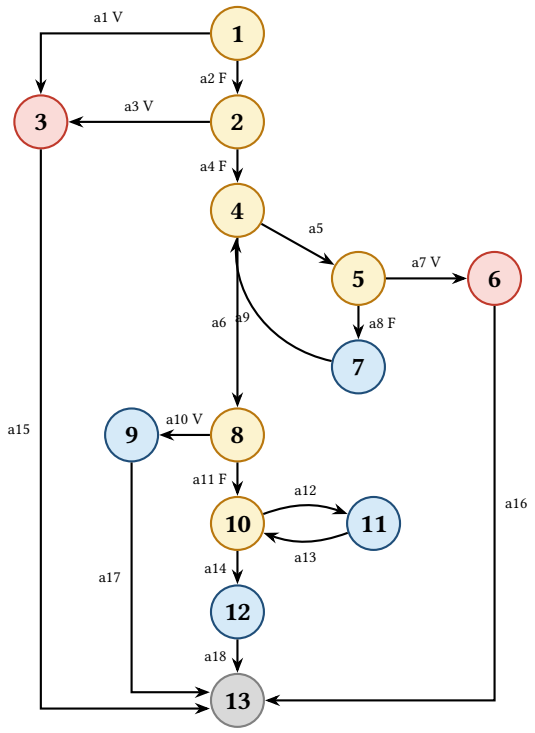
Nodo	Líneas	Contenido
1	2	D1: `pedido == null`
2	2	D2: `items.isEmpty()`
3	3	`throw IllegalArgumentException`
4	6	D3: condición del primer `for`
5	7–8	`codigo = ...` y D4: `stock < cantidad`
6	9	`throw StockInsuficienteException`
7	11	`subtotal += ...` (vuelve al nodo 4)
8	13–17	descuento, total, `cobrar(...)` y D5: `!aprobado`
9	18–20	`RECHAZADO`, notificar, `return`
10	22	D6: condición del segundo `for`
11	23	`descontarStock(...)` (vuelve al nodo 10)
12	25–30	`PAGADO`, `setTotal`, guardar, notificar, `return`
13	31	salida del método

Una decisión de modelado

La línea 5 (`subtotal = 0`) se ejecuta una sola vez, antes del bucle, y no contiene decisiones. Si se la representa como un nodo propio entre 2 y 4, se añaden un nodo y un arco, y $E - N + 2$ no cambia. Por simplicidad se la considera parte del arco $2 \to 4$.

10.1.3 Paso 3: dibujar el grafo y listar las aristas

Cada posible "salto" de un nodo a otro es una arista. Se recorre el código y se anotan:



Arista	De → a	Significado
a1	1 → 3	D1 V: pedido nulo
a2	1 → 2	D1 F
a3	2 → 3	D2 V: sin items
a4	2 → 4	D2 F
a5	4 → 5	D3: hay item por validar
a6	4 → 8	D3: no quedan items
a7	5 → 6	D4 V: sin stock
a8	5 → 7	D4 F: hay stock
a9	7 → 4	siguiente iteración
a10	8 → 9	D5 V: rechazado
a11	8 → 10	D5 F: aprobado
a12	10 → 11	D6: hay item por descontar
a13	11 → 10	siguiente iteración
a14	10 → 12	D6: no quedan items
a15	3 → 13	`throw`
a16	6 → 13	`throw`
a17	9 → 13	`return`
a18	12 → 13	`return`

Figura 10.1: Grafo de flujo de `procesarPedido`

10.1.4 Paso 4: calcular la complejidad ciclomática

Se calcula de tres formas para verificar que el grafo está bien construido. Si los tres resultados no coincidieran, el grafo tendría un error.

$V(G) = E - N + 2 = 18 - 13 + 2 = \mathbf{7}$ (se cuentan las 18 aristas y los 13 nodos)

$V(G) = P + 1 = 6 + 1 = \mathbf{7}$ (nodos predicado: 1, 2, 4, 5, 8, 10)

$V(G) = R = \mathbf{7}$ (6 regiones cerradas + la región exterior)

Interpretación: hay 7 caminos linealmente independientes, así que se necesitan **7 casos de prueba**.

10.1.5 Paso 5: derivar los caminos (método del camino base)

Se usa el método de McCabe:

1. Se elige un **camino base**: el más "normal", el flujo básico del CU-01 con un solo item.
2. Se **invierte una decisión a la vez** respecto del camino base, y cada inversión genera un camino nuevo que recorre al menos una arista nueva.

Tabla 10.3: Derivación de los caminos independientes

Camino	Cómo se obtuvo	Secuencia de nodos	Aristas nuevas
C6	**camino base** (1 item, pago aprobado)	1-2-4-5-7-4-8-10-11-10-12-13	a2, a4, a5, a6, a8, a9, a11–a14, a18
C1	invertir D1 (pedido nulo)	1-3-13	a1, a15
C2	invertir D2 (sin items)	1-2-3-13	a3
C3	invertir D4 en la 1.ª iteración	1-2-4-5-6-13	a7, a16
C5	invertir D5 (rechazado)	1-2-4-5-7-4-8-9-13	a10, a17
C4	D3 itera 2 veces y D4 se invierte en la 2.ª	1-2-4-5-7-4-5-6-13	(ninguna: 2 iteraciones del 1.er bucle)
C7	D3 y D6 iteran 3 veces	1-2-4-(5-7-4)×3-8-10-(11-10)×3-12-13	(ninguna: 3 iteraciones de cada bucle)

⚠ Razonamiento sobre C4 y C7

Con los 5 primeros caminos (C6, C1, C2, C3, C5) ya se recorren **las 18 aristas**: eso es 100 % de cobertura de ramas. Pero $V(G) = 7$ pide 2 caminos más. La inversión que faltaría, "D6 falso desde el principio" (el segundo bucle con *cero* iteraciones), es un **camino infactible**: si D2 dejó pasar el pedido, tiene al menos un item, y el segundo bucle recorre los mismos items. Los 2 caminos restantes se obtienen **variando el número de iteraciones**: C4 hace 2 vueltas del primer bucle y C7 hace 3 vueltas de cada bucle. Como vectores de aristas, son linealmente independientes de los anteriores, porque cambian la cantidad de veces que se recorren a5, a8, a9, a12 y a13. Además, cumplen la **prueba de bucles** (1 y n iteraciones).

10.1.6 Paso 6: sensibilizar cada camino (¿qué datos lo obligan?)

Sensibilizar un camino es encontrar los datos que obligan al programa a recorrerlo. Para cada decisión se pregunta: ¿de qué depende su valor?

Tabla 10.4: ¿De qué depende cada decisión?

Decisión	Depende de	Cómo la forzamos
D1, D2	el parámetro `pedido`	datos: `null` o `List.of()`
D3, D6	cantidad de items	datos: 1, 2 o 3 items
D4	respuesta de `InventarioRepository`	**stub** de `stockDisponible`
D5	respuesta de `PasarelaPago`	**stub** de `cobrar`

Tabla 10.5: Valores de cada decisión y datos que los producen

Camino	D1	D2	D3	D4	D5	D6	Datos y stubs que lo fuerzan
C1	V	–	–	–	–	–	`pedido = null`
C2	F	V	–	–	–	–	pedido sin items
C3	F	F	1 vuelta	V	–	–	P1 × 5; stub stock(P1) = 4
C4	F	F	2 vueltas	F, V	–	–	P1 × 1, P2 × 3; stock(P1) = 10, stock(P2) = 0
C5	F	F	1 vuelta	F	V	–	P1 × 2; stock = 2; stub `cobrar` → rechazado
C6	F	F	1 vuelta	F	F	1 vuelta	P3 × 1; stock = 1; stub `cobrar` → aprobado
C7	F	F	3 vueltas	F, F, F	F	3 vueltas	P1 × 1, P2 × 5, P3 × 2; stock = 100; aprobado

¿Por qué en caja blanca se simula también la calculadora?

En estas pruebas `CalculadoraDescuentos` es un `@Mock` con un valor fijo (p. ej., `calcularDescuento(20.0, REGULAR)` → `0.0`). Así el camino de `procesarPedido` no depende de que la calculadora sea correcta: si falla una prueba, el error está en `procesarPedido`. La calculadora se prueba por separado (casos CD y K).

10.1.7 Paso 7: ¿cómo se demuestra que se recorrió el camino?

Una prueba de caja blanca no solo debe dar el resultado correcto: debe **demostrar** por dónde pasó. Para eso se asigna a cada nodo una **evidencia observable** y se comprueba con `verify`:

Tabla 10.6: Evidencia observable de cada nodo

Nodo	Evidencia y cómo se comprueba
3	Se lanza `IllegalArgumentException` → `assertThrows`; ningún mock usado → `verifyNoInteractions`
5	Se consultó el stock de ese producto → `verify(inventario).stockDisponible("P1")`
6	`StockInsuficienteException` con el código del producto → `assertThrows` + mensaje
8	Se cobró el total calculado → stub con el monto exacto
9	Notificación "Pago rechazado" → `verify(notificador).enviar(EMAIL, "Pago rechazado: ...")`
11	Se descontó stock → `verify(inventario, times(n)).descontarStock(...)`
no 11	No se descontó stock → `verify(inventario, never()).descontarStock(...)`
12	Se guardó y se confirmó → `verify(notificador).enviar(EMAIL, "Pedido ... confirmado ...")`
no 12	No se guardó → `verify(pedidoRepository, never()).guardar(any())`

10.1.8 Paso 8: fichas de los casos

Productos: `P1` Cuaderno (10 Bs), `P2` Lapicero (2 Bs), `P3` Mochila (80 Bs). Correo: `EMAIL = est@umss.edu.bo`.

CB-C1 – Pedido nulo

Origen / técnica	Caja blanca, camino C1 = 1-3-13 (D1 = V).
Precondiciones	Servicio con 5 mocks (incluida la calculadora).
Datos de entrada	`pedido = null`, REGULAR.
Dobles (stubs)	Ninguno.
Pasos	Llamar a `procesarPedido(null, REGULAR)`.
Resultado esperado	`IllegalArgumentException`.
Verificaciones	`assertThrows`; `verifyNoInteractions` en los 5 mocks (no se pasó del nodo 1).
Resultado obtenido	Coincide con lo esperado. **APROBADO** ✔

CB-C2 – Pedido sin items

Origen / técnica	Caja blanca, camino C2 = 1-2-3-13 (D1 = F, D2 = V).
Precondiciones	Igual que C1.
Datos de entrada	Pedido `X` con `List.of()`.
Dobles (stubs)	Ninguno.
Pasos	Llamar a `procesarPedido`.
Resultado esperado	`IllegalArgumentException`.
Verificaciones	`assertThrows`; `verifyNoInteractions` en los 5 mocks.
Resultado obtenido	Coincide con lo esperado. **APROBADO** ✔

CB-C3 – Sin stock en el primer item

Origen / técnica	Caja blanca, camino C3 = 1-2-4-5-6-13 (D4 = V en la 1.ª vuelta).
Precondiciones	Igual que C1.
Datos de entrada	Pedido con `P1` × 5.
Dobles (stubs)	`stockDisponible("P1")` → 4 ($4 < 5$).
Pasos	Llamar a `procesarPedido`.
Resultado esperado	`StockInsuficienteException`.
Verificaciones	`verifyNoInteractions(pasarela, calculadora, pedidoRepository, notificador)`; `never().descontarStock`: no se llegó al nodo 8.
Resultado obtenido	Coincide con lo esperado. **APROBADO** ✔

CB-C4 – Sin stock en el segundo item

Origen / técnica	Caja blanca, camino C4 = 1-2-4-5-7-4-5-6-13 (2 vueltas; D4 = F y luego V).
Precondiciones	Igual que C1.
Datos de entrada	Pedido con `P1` × 1 y `P2` × 3.
Dobles (stubs)	`stockDisponible("P1")` → 10; `stockDisponible("P2")` → 0.
Pasos	Llamar a `procesarPedido`.
Resultado esperado	`StockInsuficienteException` con mensaje "…producto P2".
Verificaciones	Mensaje exacto; `verify` de `stockDisponible("P1")` y `("P2")` (2 vueltas); `verifyNoInteractions(pasarela)`.
Resultado obtenido	Coincide con lo esperado. **APROBADO** ✔

CB-C5 – Pago rechazado

Origen / técnica	Caja blanca, camino C5 = 1-2-4-5-7-4-8-9-13 (D5 = V).
Precondiciones	Igual que C1.
Datos de entrada	Pedido con `P1` × 2: subtotal 20 Bs; REGULAR.
Dobles (stubs)	`stockDisponible("P1")` → 2; `calcularDescuento(20.0, REGULAR)` → 0.0; `cobrar(EMAIL, 20.0)` → `rechazado("Sin fondos")`.
Pasos	Llamar a `procesarPedido`.
Resultado esperado	Estado `RECHAZADO`.
Verificaciones	`verify(notificador).enviar(EMAIL, "Pago rechazado: Sin fondos")` (nodo 9); `never().descontarStock` (no nodo 11); `never().guardar` (no nodo 12).
Resultado obtenido	Coincide con lo esperado. **APROBADO** ✔

CB-C6 – Pago aprobado, un item (camino base)

Origen / técnica	Caja blanca, camino C6 = 1-2-4-5-7-4-8-10-11-10-12-13.
Precondiciones	Igual que C1.
Datos de entrada	Pedido `PED-6` con `P3` × 1: subtotal 80 Bs; VIP.
Dobles (stubs)	`stockDisponible("P3")` → 1; `calcularDescuento(80.0, VIP)` → 4.0; `cobrar(EMAIL, 76.0)` → `aprobado("TX-6")`; `guardar(pedido)` → `pedido`.
Pasos	Llamar a `procesarPedido`.
Resultado esperado	Estado `PAGADO`; total $80 - 4 = 76$; se devuelve el pedido guardado.
Verificaciones	`verify(inventario).descontarStock("P3", 1)` (nodo 11); mensaje exacto "Pedido PED-6 confirmado. Total: 76.0 Bs. Transacción: TX-6" (nodo 12).
Resultado obtenido	Coincide con lo esperado. **APROBADO** ✔

CB-C7 – Pago aprobado, tres items

Origen / técnica	Caja blanca, camino C7 (3 vueltas en cada bucle); prueba de bucles.
Precondiciones	Igual que C1.
Datos de entrada	`P1` × 1, `P2` × 5, `P3` × 2: subtotal $10 + 10 + 160 = 180$; REGULAR.
Dobles (stubs)	`stockDisponible(anyString())` → 100; `calcularDescuento(180.0, REGULAR)` → 9.0; `cobrar(EMAIL, 171.0)` → `aprobado("TX-7")`; `guardar` → `pedido`.
Pasos	Llamar a `procesarPedido`.
Resultado esperado	Se valida y descuenta cada uno de los 3 productos.
Verificaciones	`times(3)` en `stockDisponible` y `descontarStock`; `descontarStock` con (P1, 1), (P2, 5) y (P3, 2); una sola confirmación.
Resultado obtenido	Coincide con lo esperado. **APROBADO** ✔

10.1.9 Paso 9: código de un caso, anotado con los nodos

```
@Test
@DisplayName("C5: pago rechazado")
void c5PagoRechazado() {
    // DATOS -> D1 = F (no es null), D2 = F (tiene 1 item), D3 = 1 vuelta
    Pedido pedido = new Pedido("X", EMAIL, List.of(new ItemPedido(P1, 2)));
    // STUB -> nodo 5: D4 = F (stock 2, no es menor que 2)
    when(inventario.stockDisponible("P1")).thenReturn(2);
    // STUB -> nodo 8: aislar la calculadora con un valor fijo
    when(calculadora.calcularDescuento(20.0, TipoCliente.REGULAR)).thenReturn(0.0);
    // STUB -> nodo 8: D5 = V (el banco rechaza)
    when(pasarela.cobrar(EMAIL, 20.0)).thenReturn(ResultadoPago.rechazado("Sin fondos"));

    Pedido resultado = servicio.procesarPedido(pedido, TipoCliente.REGULAR);

    assertEquals(EstadoPedido.RECHAZADO, resultado.getEstado());          // nodo 9
    verify(notificador).enviar(EMAIL, "Pago rechazado: Sin fondos");      // nodo 9
    verify(inventario, never()).descontarStock(anyString(), anyInt());    // NO pasó por 11
    verify(pedidoRepository, never()).guardar(any());                     // NO pasó por 12
}
```

Código 10.1: Camino C5: cada línea de la prueba se relaciona con un nodo del grafo

10.1.10 Paso 10: ejecutar y medir la cobertura

```
Caja blanca - PedidoService.procesarPedido (camino básico) ✔
|- C1: pedido null ✔
|- C2: pedido sin items ✔
|- C3: stock insuficiente en el primer item ✔
|- C4: el bucle itera una vez y falla en el segundo item ✔
|- C5: pago rechazado ✔
|- C6: pago aprobado con un item ✔
`- C7: pago aprobado con tres items (bucle de descuento itera 3 veces) ✔
```

Código 10.2: Salida de la ejecución (ordenada)

Después se mide con JaCoCo (`./run.sh cobertura`) y se lee el reporte **línea por línea**. JaCoCo cuenta las ramas del *bytecode*; por eso la línea 42 del archivo fuente, con la condición compuesta `pedido == null || isEmpty()`, tiene 4 ramas (2 por cada condición simple):

Tabla 10.7: Ramas cubiertas por línea de `PedidoService.java` (datos reales de JaCoCo)

Línea	Código	Solo caja negra	Solo caja blanca
42	`if (pedido == null || ...isEmpty())`	3/4	4/4
47	`for` (validación)	2/2	2/2
49	`if (stock < cantidad)`	2/2	2/2
59	`if (!pago.aprobado())`	2/2	2/2
65	`for` (descuento de stock)	2/2	2/2
	Total	**11/12**	**12/12**

Lectura: en el reporte HTML, JaCoCo pinta la línea 42 en amarillo (rama parcial) cuando solo se ejecuta la caja negra. Al pasar el cursor indica "1 of 4 branches missed": la rama `pedido == null` verdadera. Esa rama corresponde al camino C1, que solo apareció al analizar el código. Es el hallazgo OBS-01 de la SRS.

10.2 `calcularDescuento`: cobertura de decisiones

10.2.1 Pasos 1 y 2: código numerado y nodos

```
1  public double calcularDescuento(double subtotal, TipoCliente
       tipo) {
2      if (subtotal < 0) {                          // d1
3          throw new IllegalArgumentException(...);
4      }
5      double porcentaje;
6      if (subtotal >= 500) {                       // d2
7          porcentaje = 0.10;
8      } else if (subtotal >= 100) {                // d3
9          porcentaje = 0.05;
10     } else {
11         porcentaje = 0.0;
12     }
13     if (tipo == TipoCliente.VIP) {               // d4
14         porcentaje += 0.05;
15     }
16     return Math.round(subtotal * porcentaje * 100.0) / 100.0;
17 }
```

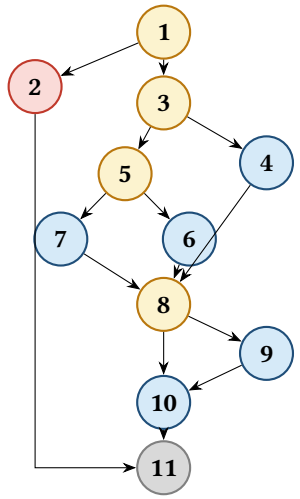


Figura 10.2: Grafo de `calcularDescuento`

Nodos: 1 = d1 (línea 2); 2 = `throw` (3); 3 = d2 (5–6); 4 = 10 % (7); 5 = d3 (8); 6 = 5 % (9); 7 = 0 % (11); 8 = d4 (13); 9 = +5 % (14); 10 = `return` (16); 11 = salida.

10.2.2 Pasos 3 y 4: aristas y complejidad

Aristas: 1→2, 1→3, 3→4, 3→5, 5→6, 5→7, 4→8, 6→8, 7→8, 8→9, 8→10, 9→10, 2→11, 10→11, es decir, $E = 14$ y $N = 11$.

$$V(G) = 14 - 11 + 2 = 5 \qquad V(G) = P + 1 = 4 + 1 = 5$$

10.2.3 Paso 5: caminos por inversión de decisiones

Tabla 10.8: Caminos de `calcularDescuento`, obtenidos invirtiendo una decisión a la vez

Camino	Obtenido	Nodos	d1	d2	d3	d4	Entrada	Esperado
K2	base	1-3-4-8-10-11	F	V	–	F	(600, REG)	60,00
K1	invertir d1	1-2-11	V	–	–	–	(−1, REG)	error
K5	invertir d4	1-3-4-8-9-10-11	F	V	–	V	(600, VIP)	90,00
K4	invertir d2	1-3-5-7-8-10-11	F	F	F	F	(20, REG)	0,00
K3	invertir d3 (desde K4)	1-3-5-6-8-9-10-11	F	F	V	V	(200, VIP)	20,00

Con estos 5 casos, cada decisión toma los valores V y F al menos una vez: 8 de 8 ramas (100 %). El oráculo se calcula igual que en caja negra; por ejemplo, K3: $200 \times (0{,}05 + 0{,}05) = 20{,}00$.

10.3 Comparación de los dos enfoques

Tabla 10.9: Caja negra frente a caja blanca en este proyecto

	Caja negra	**Caja blanca**
Base de prueba	SRS: RF-01 a RF-05, RN-01 a RN-04, CU-01	Código fuente y grafo de flujo
Técnicas	Clases de equivalencia, valores límite, tabla de decisión	Camino básico (McCabe), cobertura de ramas, prueba de bucles
Rol de Mockito	**Stub**: provee entradas (stock, respuesta del banco)	**Stub + mock**: fuerza cada decisión y demuestra con `verify` por dónde pasó
Aserciones	Solo salidas (`assertEquals`, `assertThrows`)	Salidas + `verify`, `never`, `times`, `verifyNoInteractions`
Robustez ante refactorización	Alta	Menor (más acoplada a la implementación)
Ramas de `PedidoService`	11/12	**12/12**
Encontró	error de límite <= (caso L1)	rama no especificada `pedido == null` (OBS-01)

Capítulo 11

Resultados

11.1 Ejecución de las pruebas

```
== Ejecutando pruebas ==
|- Caja blanca - PedidoService.procesarPedido (camino básico)   7/7  OK
|- Caja blanca - CalculadoraDescuentos (cobertura de ramas)     5/5  OK
|- Caja negra - CalculadoraDescuentos                          15/15 OK
|- Caja negra - PedidoService (tabla de decisión)               5/5  OK
|- E01 ... E10 (API y arquitectura)                            30/30 OK
|- Paso1, Paso4, Paso5 (construcción paso a paso)               3/3  OK
|- Paso2, Paso3, Paso6 (rojos a propósito, @Disabled)           3 omitidas
Test run finished after 1332 ms
[        68 tests found           ]
[         3 tests skipped         ]
[        65 tests successful      ]
[         0 tests failed          ]
```

Código 11.1: Ejecución con `./run.sh cobertura` (resumen)

Tabla 11.1: Resumen de pruebas por categoría

Categoría	Pruebas	Resultado
Ejemplos de la API y arquitectura (`basico`)	30	30 ✔
Caja negra (`cajanegra`)	20	20 ✔
Caja blanca (`cajablanca`)	12	12 ✔
Construcción paso a paso (`pasoapaso`, en verde)	3	3 ✔
Total	**65**	**65** ✔

\+ 3 pruebas en rojo a propósito (`./run.sh rojos`), que fallan como se documenta en el capítulo 8.

11.2 Cobertura (JaCoCo 0.8.15)

Tabla 11.2: Cobertura de la capa de servicio

Clase	Instrucciones	Ramas	Líneas	Complejidad
`PedidoService`	155/155 (100 %)	12/12 (100 %)	35/35	10/10
`CalculadoraDescuentos`	45/45 (100 %)	8/8 (100 %)	11/11	6/6

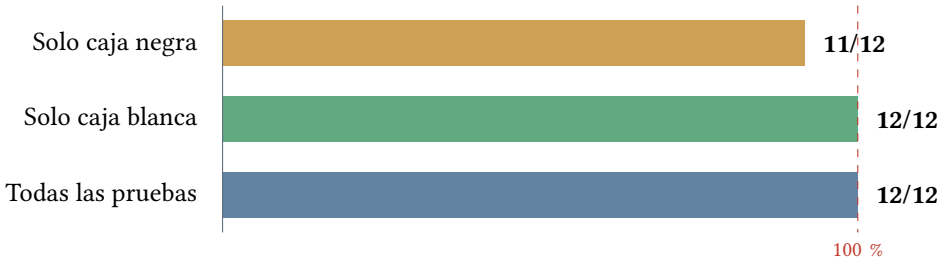


Figura 11.1: Ramas cubiertas de `PedidoService` según el tipo de prueba

Hallazgo

Las pruebas de caja negra, diseñadas solo a partir de los requisitos, cubren 11 de 12 ramas de `PedidoService`. La rama `pedido == null` no aparece en la tabla de decisión porque la SRS no habla de "pedidos nulos". Solo al analizar el código (caja blanca, camino C1) se detectó que esa rama existía y estaba sin probar; se registró como hallazgo OBS-01 de la SRS. A la inversa, el caso de valor límite L1 de la caja negra detectó el error de mutación <=. **Las dos técnicas son complementarias**: la caja negra comprueba que se hace lo que se pidió y la caja blanca, que todo el código escrito está probado.

Capítulo 12

Conclusiones y recomendaciones

12.1 Conclusiones

1. Mockito permite escribir pruebas unitarias **rápidas, aisladas y repetibles**: las 65 pruebas del proyecto se ejecutan en poco más de un segundo sin base de datos, banco ni servidor de correo.
2. Mockito tiene una arquitectura en capas: una fachada pública (`Mockito`), un núcleo interno (`MockHandler`, `InvocationContainer`, `MockingProgress`, `VerificationMode`...), un sistema de plugins y bibliotecas de generación de código (Byte Buddy, Objenesis). Conocer esos componentes explica reglas prácticas como "todos los argumentos con matchers o ninguno" o el uso de `doXxx()` en métodos `void`, y permite diagnosticar cualquier error de Mockito.
3. Las pruebas **se diseñan antes de programarse**. Seguimos un proceso explícito: SRS → condiciones → elementos de cobertura → fichas de casos con su oráculo calculado a mano → código. Así, cada prueba se justifica y se puede auditar.
4. La testabilidad **se diseña**. Especificar los requisitos en una SRS verificable y documentar el diseño en un SDD con interfaces e inyección por constructor fue lo que hizo posible reemplazar cada dependencia por un mock y trazar cada requisito hasta sus pruebas.
5. Diseñar las pruebas también **audita los requisitos**: encontramos 4 vacíos en la SRS (OBS-01 a OBS-04).
6. En **caja negra**, Mockito actúa como *stub* que provee entradas. En **caja blanca**, fuerza cada rama del grafo de flujo y demuestra con `verify` qué se ejecutó.
7. Se alcanzó el 100 % de cobertura de ramas en la capa de servicio y se cumplió el RNF-02. La comparación de coberturas mostró que ninguna técnica por separado basta.
8. Para la **auditoría de sistemas**, un conjunto de pruebas automatizadas con matriz de trazabilidad y reporte de cobertura es evidencia objetiva y verificable de la calidad del software.

12.2 Recomendaciones

- Integrar las pruebas en un pipeline de integración continua (GitHub Actions, GitLab CI) y bloquear los cambios que las rompan o bajen la cobertura.
- Complementar las pruebas unitarias con pruebas de integración (p. ej., Testcontainers) para validar que las dependencias reales se comportan como los mocks suponen.
- Evitar el exceso de `verify()`: verificar solo las interacciones que forman parte del contrato.
- Como auditores, pedir el reporte de cobertura y la matriz de trazabilidad, y volver a ejecutar las pruebas para confirmar los resultados.

Referencias

[1] Mockito. *Mockito framework site*. https://site.mockito.org

[2] Mockito. *Mockito Javadoc (5.x)*. https://javadoc.io/doc/org.mockito/mockito-core/latest/org/mockito/Mockito.html

[3] Mockito. *Repositorio oficial*. https://github.com/mockito/mockito

[4] JUnit Team. *JUnit User Guide*. https://docs.junit.org

[5] Meszaros, G. (2007). *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley.

[6] Fowler, M. (2007). *Mocks Aren't Stubs*. https://martinfowler.com/articles/mocksArentStubs.html

[7] McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320.

[8] Pressman, R. S. y Maxim, B. R. (2020). *Ingeniería del software: un enfoque práctico* (9.ª ed.). McGraw-Hill.

[9] Myers, G. J., Sandler, C. y Badgett, T. (2011). *The Art of Software Testing* (3.ª ed.). Wiley.

[10] IEEE (2009). *IEEE Std 1016-2009 – Standard for Information Technology – Systems Design – Software Design Descriptions*.

[11] ISO/IEC/IEEE 29119 (2013–2022). *Software and systems engineering – Software testing*.

[12] ISACA (2018). *COBIT 2019 Framework: Governance and Management Objectives*.

[13] Freeman, S. y Pryce, N. (2009). *Growing Object-Oriented Software, Guided by Tests*. Addison-Wesley.

[14] JaCoCo. *Java Code Coverage Library*. https://www.jacoco.org/jacoco

Capítulo A

Estructura del proyecto y ejecución

```
ejemplos-mockito/
|-- pom.xml                     # Maven: JUnit 6, Mockito 5.24, JaCoCo
|-- run.sh                      # Ejecuta todo sin Maven (solo JDK 21+)
|-- README.md
`-- src/
    |-- main/java/bo/edu/umss/pedidos/
    |   |-- modelo/      Pedido, ItemPedido, Producto, ResultadoPago, enums, excepciones
    |   |-- puertos/     InventarioRepository, PasarelaPago, Notificador, PedidoRepository
    |   `-- servicio/    PedidoService, CalculadoraDescuentos
    `-- test/java/bo/edu/umss/pedidos/
        |-- basico/      E01 ... E10  (API y arquitectura de Mockito)
        |-- pasoapaso/   Paso1 ... Paso6 (construcción de una prueba, de rojo a verde)
        |-- cajanegra/   CalculadoraDescuentosCajaNegraTest, PedidoServiceCajaNegraTest
        `-- cajablanca/  PedidoServiceCajaBlancaTest, CalculadoraDescuentosCajaBlancaTest
```

Código A.1: Estructura de `ejemplos-mockito/`

```
cd ejemplos-mockito
./run.sh               # compila y ejecuta las pruebas (65 verdes + 3 omitidas)
./run.sh cobertura     # + reporte HTML en target/cobertura/index.html
./run.sh rojos         # ejecuta las 3 versiones en rojo del paso a paso
mvn test               # alternativa con Maven (reporte en target/site/jacoco)
```

Código A.2: Comandos de ejecución